SYSTEM OWNER'S HANDBOOK

Cross-Cutting Concerns (Part 7)

Multi-tenant clinic management platform

Status as of 2026-10-28 · code is the source of truth · written for a software-engineer owner

How to read this document

This is part of the **System Owner's Handbook** for a multi-tenant clinic management product, called "the Platform" throughout. It is written for the product owner, a software engineer, and every chapter can be read on its own.

Status labels. Every feature carries exactly one label, based on the code as of 2026-10-28:

- **[Implemented]**: built end to end.
- **[Partial]**: some of it exists; the text says exactly what is missing.
- **[Not implemented]**: planned, seeded, disabled or only mentioned. It is never described as working.
- **[Unverified]**: the code was read but the behaviour could not be confirmed; the text says what is unconfirmed.

Source of truth. The code is the source of truth. Where older documentation disagreed with the code, the handbook follows the code. Nothing was executed while writing: behaviour is described from reading the code.

Figures. Every figure has a number, a caption and a paragraph that describes it in words. Figures captioned "screenshot pending" are placeholders; the paragraph after them describes the screen from the code. The few real screenshots were taken on 2026-10-23 and show a former working name of the product.

Names. "Organization" is one clinic business (the tenant); "location" is one of its branches. File paths and code names appear in a short "Technical references" line at the end of each feature. Example people and places are fictional.

Table of contents

Part 7 — Cross-cutting concerns

Chapter 56 — Authentication and sessions 4
Chapter 57 — Access control in depth 18
Chapter 58 — Clinic customization 27
Chapter 59 — Notification mechanics 36
Chapter 60 — Audit logging 41
Chapter 61 — Error handling 46
Chapter 62 — Time, money and document numbers 53
Chapter 63 — Realtime updates and background jobs 58

Chapter 56 — Authentication and sessions

This chapter explains how a person proves who they are to the Platform and stays signed in: registering a clinic, signing in, choosing an organization and location, the short-lived access token and the long-lived session cookie, managing sessions, password flows, rate limiting and the staff-login administration screen. It is written for the product owner, who needs to know exactly how long a sign-in lasts, what a user can revoke and which protections exist today. What a signed-in person may then do is a different question, answered by Chapter 11 (roles and permissions) and Chapter 57 (access control in depth).

56.1 The model in one paragraph

Status: [Implemented]

A **user** is a login: an email address, a password hash and an account status (`active` , `suspended` or `deleted`). A user is not the same as an **employee**, who is the person record that the Employees module keeps (Chapter 37). A **staff login** links one user to one employee. Signing in creates a **session** row in the database and gives the browser two things. The first is an **access token**, valid for 15 minutes, which the web app keeps in memory only and sends as a Bearer header. The second is a **session cookie** named `refresh_token` , which is `httpOnly` (scripts cannot read it) and carries the session id and a long random secret. When the access token expires, the web app calls `POST /auth/refresh` ; the cookie proves the session is still valid and a new access token is issued. Nothing about the login is stored in the browser's local storage.

Technical references: `backend/src/auth/auth.service.ts` · `AuthService` · `backend/src/session/session.service.ts` · `SessionService` · `backend/src/auth/jwt.service.ts` · `web-app/src/store/useAuthStore.ts`

56.2 Screens

The sign-in related screens are public pages (`/login` , `/register` , `/forgot-password` , `/reset-password` , `/verify-email`), the location selector (`/select-location`), the signed-in pages for sessions and security (`/sessions` and the Security and Sessions sections of Settings), and the staff-login administration page (`/users`).

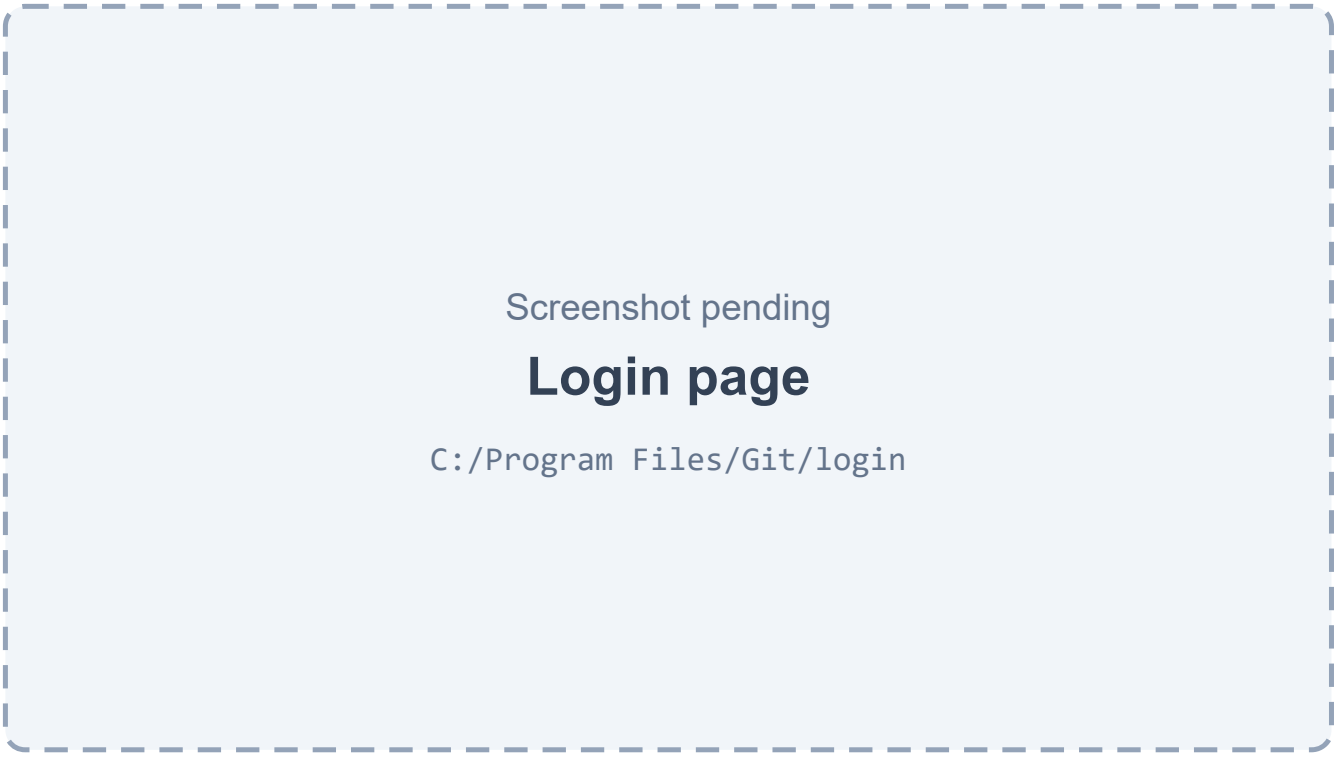


Figure 56.1 — Login page (screenshot pending).

The login page shows the clinic's own name and logo when the browser knows which clinic it last used (Chapter 58 explains the public theme). It has an email field, a password field with a show and hide button, a "Keep me signed in for 30 days" check box, a "Forgot password?" link, a sign-in button and a link to the sign-up page. The sign-up page asks for clinic name, the owner's name, email, an optional phone number and a password, with a five-segment strength meter (Weak, Fair, Good, Strong, Very strong) that is advice only and does not block the form.

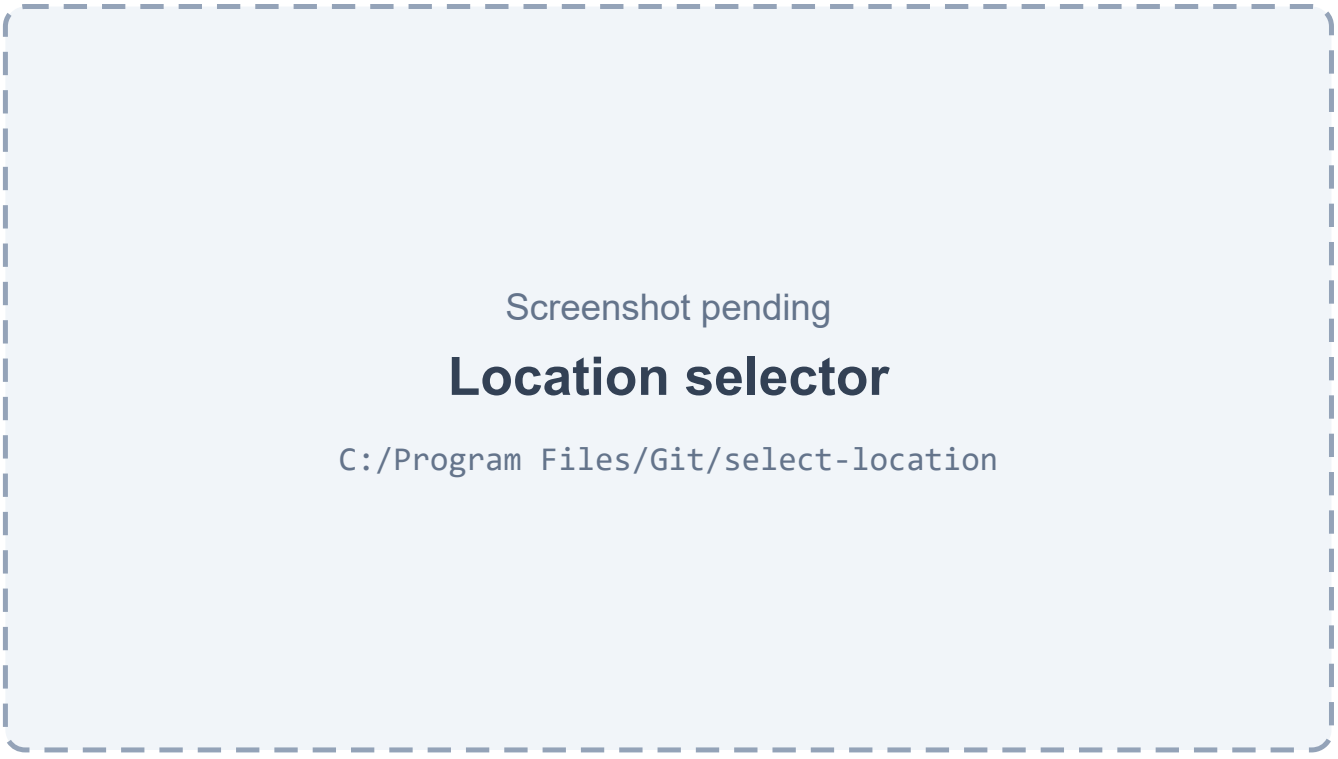


Figure 56.2 — Location selector (screenshot pending).

The selector is a two-step screen. Step one lists the organizations the user can enter; choosing one with a single location selects that location and continues at once. Step two lists the organization's locations as cards; the user picks one and continues. If an organization has no location, the screen says that a location must be created to continue.

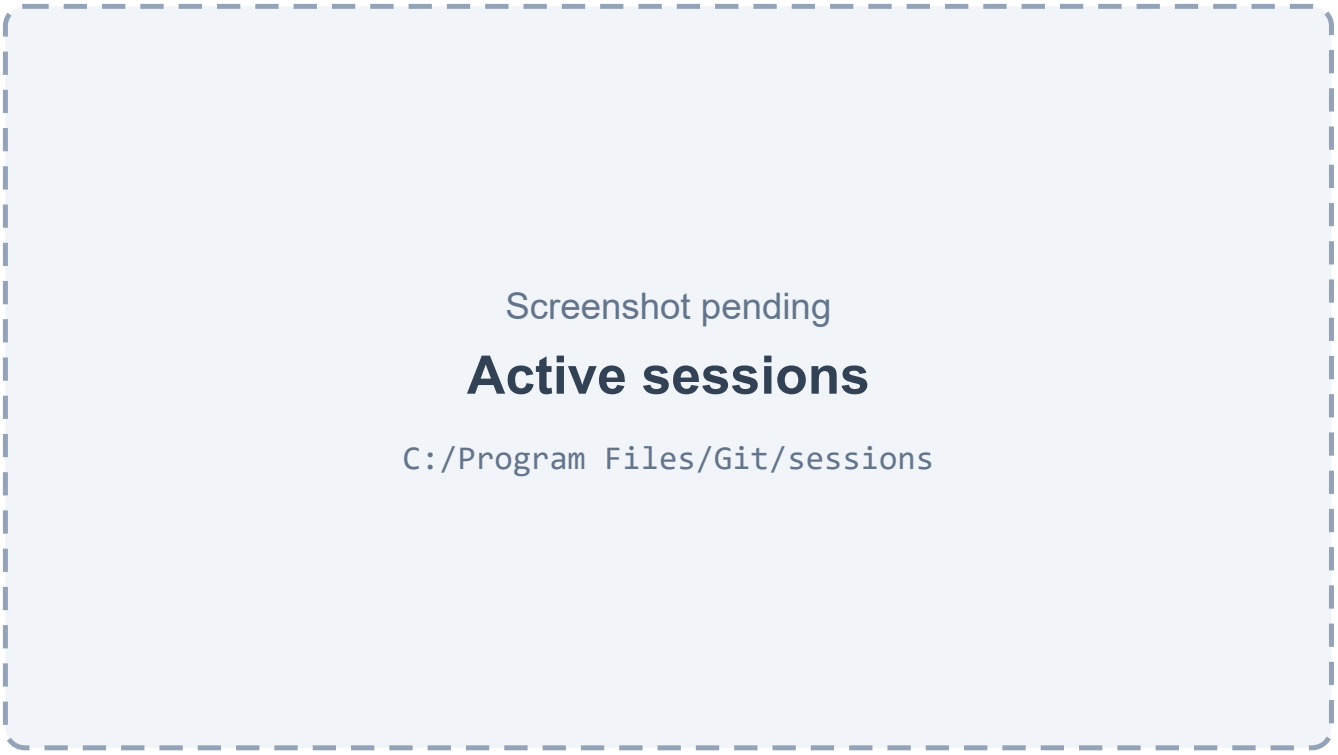


Figure 56.3 — Active sessions (screenshot pending).

The sessions list (page title "Active Sessions", also available as the Sessions section of Settings) shows one row per active session with browser, operating system, device, IP address, the time it started and the time it was last active. The current session is marked. Every other row has a **Revoke** button, the current row has **Sign out**, and a **Sign out all others** button sits at the top.

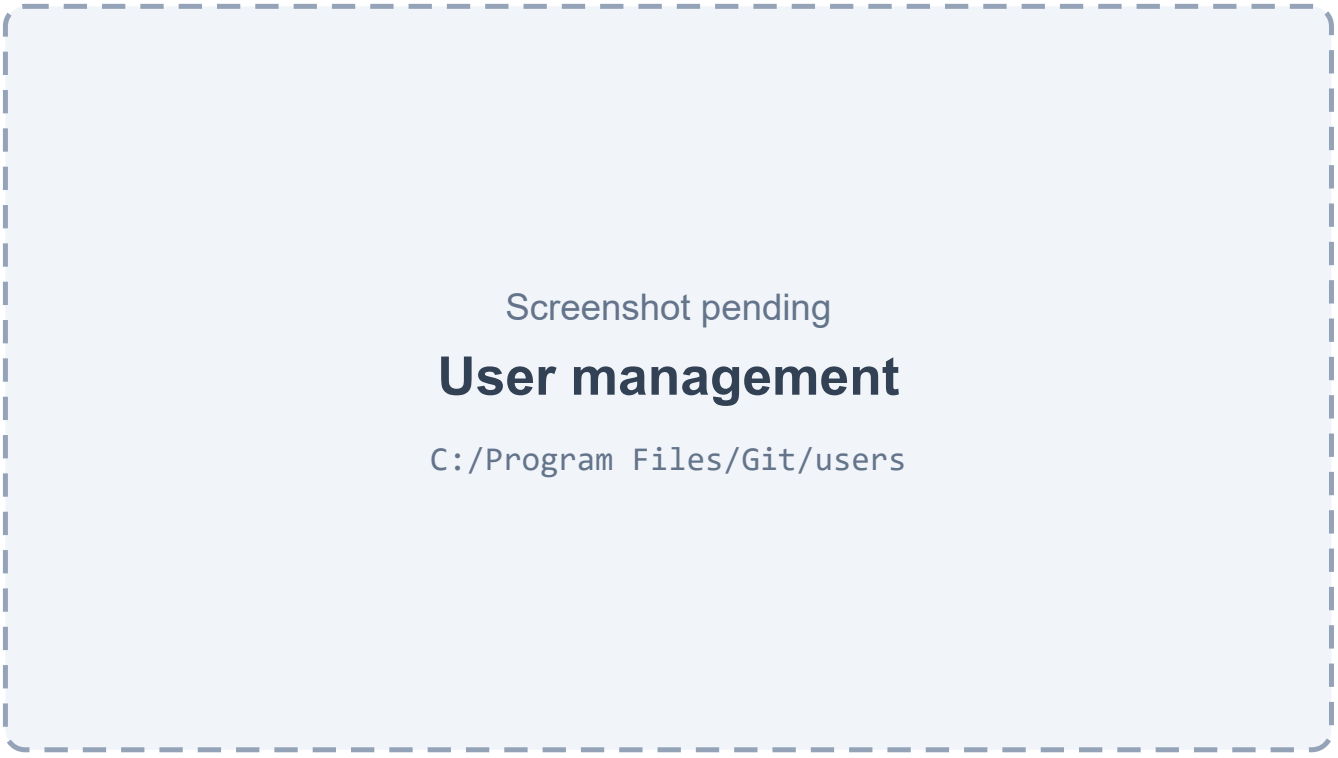


Figure 56.4 — User management (screenshot pending).

The user administration page (`/users` , title "User Management", subtitle "Manage system access across roles and organizations") lists users by type and has a **Create User** button. The create form asks for name, email, password, role type (Super Admin, Owner or Staff), and, for a staff user, which employee the login is for. Each row can be edited (including its status) or deleted.

56.3 Features

56.3.1 Register a clinic (self-signup)

Status: [Implemented]

What it does. Lets a new clinic owner create the login and the clinic in one step. The Platform creates the user, a new organization with a main location and a 14-day trial (Chapter 45 covers subscriptions and onboarding), sends a verification email and signs the owner straight in.

How the user does it.

1. Open the sign-up page and enter clinic name, name, email, optional phone and a password.
2. Submit. The owner lands in the application, kept signed in for 30 days.
3. Later, click the link in the verification email.

Behind the scenes — technically. `POST /auth/register` is public and rate limited. `AuthService.register` creates the user, calls the clinic set-up service to create the organization, and sends the verification email. If clinic creation fails, the new user row is deleted so no empty login remains. It then runs the normal login flow with "keep me signed in" on.

Rules and validations.

- Email must be a valid address; name and clinic name are required (at most 255 characters); phone at most 20 characters; password at least 6 characters.
- An email that already exists is refused.

- The verification email failing to send is logged but does not stop registration.

Who can do it. Anyone; the route is public.

Data read and written. `users`, `organizations`, `locations`, owner link, subscription rows, `user_sessions`, `security_logs`.

Error cases. 429 when more than 5 sign-ups come from one address in an hour; 400 for invalid fields or a duplicate email.

Technical references: `backend/src/auth/auth.controller.ts` · `register` · `backend/src/subscriptions/clinic-setup.service.ts` · `web-app/src/features/auth/components/RegisterForm.tsx`

56.3.2 Verify an email address

Status: [Implemented]

What it does. Confirms that the owner controls the email address. Verification is informational: an unverified user can still sign in and work. The web app shows a "verify your email" banner until verification is done.

How the user does it. Click the link in the email, which opens `/verify-email?token=…`. The page shows "Verifying your email…" and then a success or failure message. A signed-in user whose address is still unverified can ask for a fresh link.

Behind the scenes — technically. The email carries a random 32-byte token; the database stores only its SHA-256 hash, valid for 48 hours. `POST /auth/verify-email` (public, 10 attempts per 15 minutes) hashes the supplied token, finds an unexpired match and sets `email_verified_at`. `POST /auth/resend-verification` (signed in, 3 per hour) issues a new token; if the user has no pending token it answers "Email already verified". Sending depends on SMTP settings in the server environment (`SMTP_USER`, `SMTP_PASS`).

Rules and validations. An invalid or expired token gives 400 "This verification link is invalid or has expired". Users created by an administrator through user administration are not sent a verification email.

Who can do it. Anyone holding the token; resend needs a signed-in user.

Data read and written. `users` (`email_verification_token`, `email_verification_expires`, `email_verified_at`).

Technical references: `AuthService.verifyEmail` · `AuthService.resendVerification` · `backend/src/common/services/mail.service.ts` · `web-app/src/features/account/pages/VerifyEmailPage.tsx`

56.3.3 Sign in

Status: [Implemented]

What it does. Checks the email and password, creates a session and returns the information the web app needs to continue: the user, an access token, the organizations and locations the user may enter, and flags saying whether an organization or location still has to be chosen.

How the user does it. Enter email and password, optionally tick "Keep me signed in for 30 days", and click sign in.

Behind the scenes — in simple terms. The server looks up the user, compares the password against the stored hash, refuses suspended or deleted accounts, records a new session, and works out where the person is allowed to go. The session lasts one day, or 30 days if the box was ticked. Only five sessions can be active at once; a sixth sign-in quietly revokes the oldest.

Behind the scenes — technically. `POST /auth/login` (public; 5 attempts per 15 minutes per address and route). `UserService.validateUser` compares with bcrypt (12 rounds). `SessionService.createSession` stores a bcrypt hash of a 64-byte random refresh secret, the user agent, parsed browser, operating system and device, and the client IP. `AuthorizationService.resolveUserAccess` returns the accessible organizations. The response also lists any lab memberships, so a user who works only for a lab is sent to the lab portal (Chapter 41). The cookie is `httpOnly`, `sameSite=lax`, `secure` in production, with a maximum age equal to the session's remaining lifetime.

Rules and validations.

- Email valid; password at least 6 characters.
- A wrong email or password gives a plain 401 with no hint about which was wrong.
- A suspended account gets 403 "Your account has been suspended. Please contact support."; a deleted account gets 403 "This account has been deleted."
- A session's expiry is fixed at sign-in and is never extended by refreshing.

Who can do it. Anyone with an account.

Data read and written. `users`, `user_sessions`, `employee_org_location_assignments`, owner links, `organizations`, `locations`, `security_logs` (a success row).

Effects on other modules. The login result seeds the web app's auth store, which every later request reads for its tenant headers (Chapter 8).

Error cases. 401 wrong credentials; 403 suspended or deleted; 429 too many attempts (message "Too many requests. Please try again later.", with a `retryAfter` value in seconds).

Technical references: `AuthService.login` · `SessionService.createSession` · `AuthorizationService.resolveUserAccess` · `web-app/src/features/auth/components/LoginForm.tsx`

56.3.4 Choose and switch organization and location

Status: [Implemented]

What it does. Decides which clinic and branch a person is working in. Every later request carries that choice in two headers.

How the user does it. After sign-in the selector appears when a choice is needed. Later, the location switcher in the page header changes the active location.

Behind the scenes — in simple terms. A single organization with a single location is chosen automatically. A super admin always picks an organization first. Otherwise the person picks, and the Platform remembers the last choice for next time.

Behind the scenes — technically. The login response sets `requiresOrgSelection` and `requiresLocationSelection`. `POST /auth/select-location` and `POST /auth/switch-location` (both signed in, context free) validate access with `AuthorizationService.validateAccess` and save `last_accessed_organization_id` and `last_accessed_location_id` on the user. Switching also returns the legacy permission flags for the new location (deprecated; the web app reads `GET /me/access`). Chapter 8 describes the choice flow and what each type of user may enter in full.

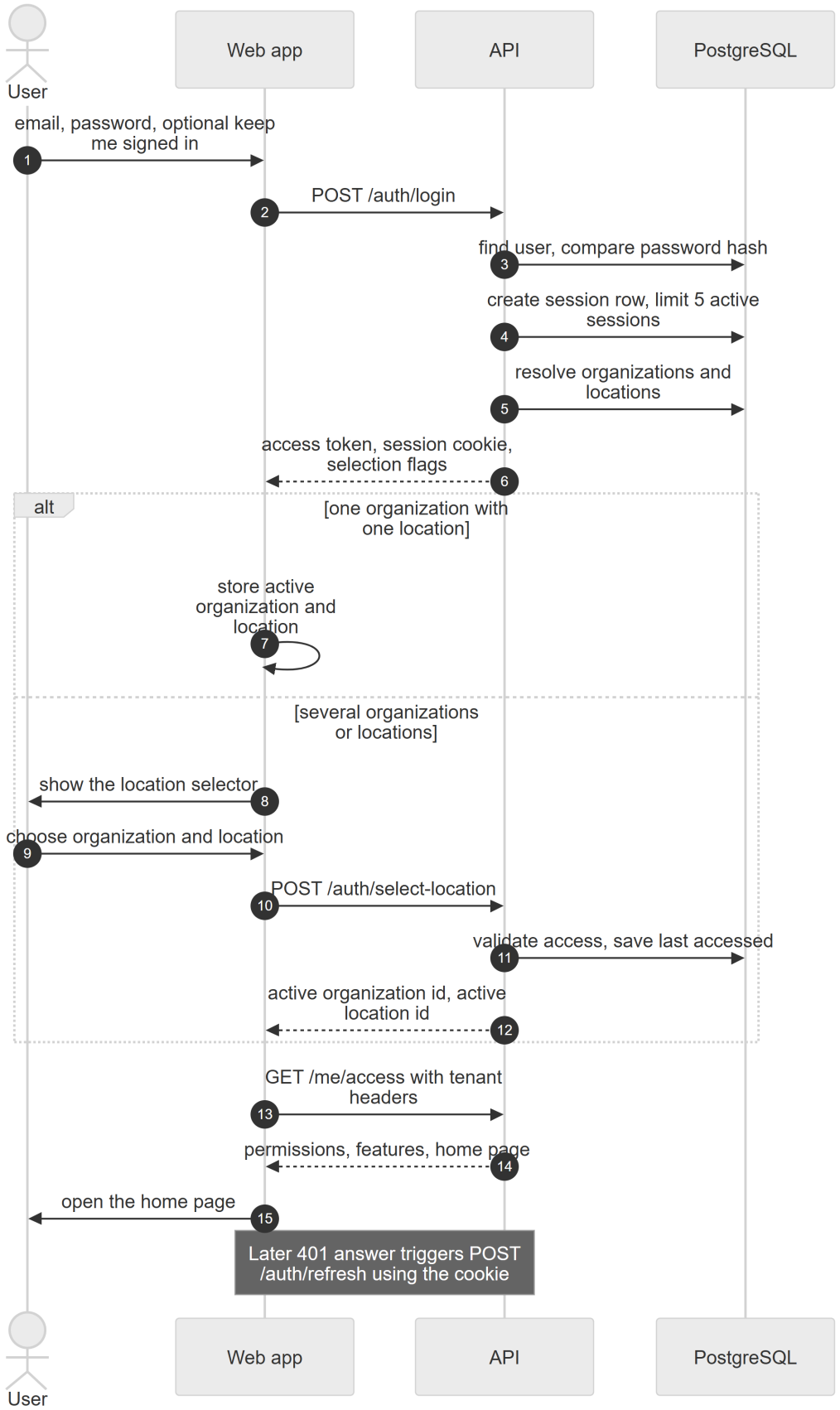


Figure 56.5 — Sequence of a sign-in, the optional location choice and the first access fetch.

The diagram has three participants besides the User: the Web app, the API and PostgreSQL. (1) The user enters email, password and the optional keep-me-signed-in choice. (2) The web app calls `POST /auth/login`. (3) The API finds the user and compares the password hash. (4) It creates the session row and enforces the limit of five active sessions. (5) It resolves the organizations and locations. (6) It answers with the access token, the session cookie and the selection flags. An "alt" box then shows two paths. If the user has one organization with one

location, the web app stores them as active. Otherwise the web app shows the selector, the user chooses, the web app calls `POST /auth/select-location`, the API validates access and saves the last accessed choice, and returns the two active ids. After the box, the web app calls `GET /me/access` with the tenant headers and receives permissions, features and the home page, and opens the home page. A note says that a later 401 answer triggers `POST /auth/refresh` with the cookie.

Rules and validations. Both organization id and location id are required UUIDs. A request for a location the user may not enter gets 403 "You do not have access to this location".

Who can do it. Any signed-in user.

Data read and written. `users.last_accessed_*`.

Error cases. 403 no access; the web app shows the toast "Failed to select location".

Technical references: `AuthService.selectLocation` · `AuthService.switchLocation` · `web-app/src/features/auth/components/LocationSelector.tsx` · `web-app/src/components/layout/LocationSwitcher.tsx` · `web-app/src/routes/__root.tsx`

56.3.5 Stay signed in: access token, refresh cookie and silent refresh

Status: [Implemented]

What it does. Keeps a person signed in across page reloads and across the 15-minute life of the access token, without storing credentials in the browser.

Behind the scenes — in simple terms. The access token is a short pass kept only in the page's memory. When the page is reloaded the memory is empty, so the web app quietly asks the server for a new pass using the cookie. When a pass expires in the middle of work, the same quiet request happens and the failed request is repeated.

Behind the scenes — technically.

- The access token is a JWT signed with the server's access secret, expiring in 15 minutes, holding the user id, email, global role and session id. There is no JWT refresh token; the refresh secret is a random value whose bcrypt hash lives on the session row.
- On every app start the root route calls `POST /auth/refresh` once ("silent refresh"). On success the auth store is filled; on failure the user is treated as signed out.
- `POST /auth/refresh` (public) reads the cookie, validates the session (not revoked, not expired, secret matches), re-checks the account status, **rotates** the refresh secret (the old one stops working) and returns a new access token and the full login payload. The cookie's expiry is the original session expiry, not a fresh window.
- When any API call returns 401, the axios interceptor refreshes and retries once (Chapter 61 shows the sequence).
- The auth store is memory only; it never uses local storage.

Rules and validations. A missing, revoked or expired session gives 403 "Invalid or expired session" and the cookie is cleared. A session whose user turned suspended or deleted is revoked on the spot.

Who can do it. Any browser holding a valid cookie.

Data read and written. `user_sessions` (`refresh_token_hash`, `last_active_at`).

Technical references: `AuthService.refreshTokens` · `SessionService.rotateRefreshToken` · `JwtTokenService` · `backend/src/session/session-lifetime.ts` · `web-app/src/lib/api.ts`

56.3.6 Sign out, list and revoke sessions

Status: [Implemented]

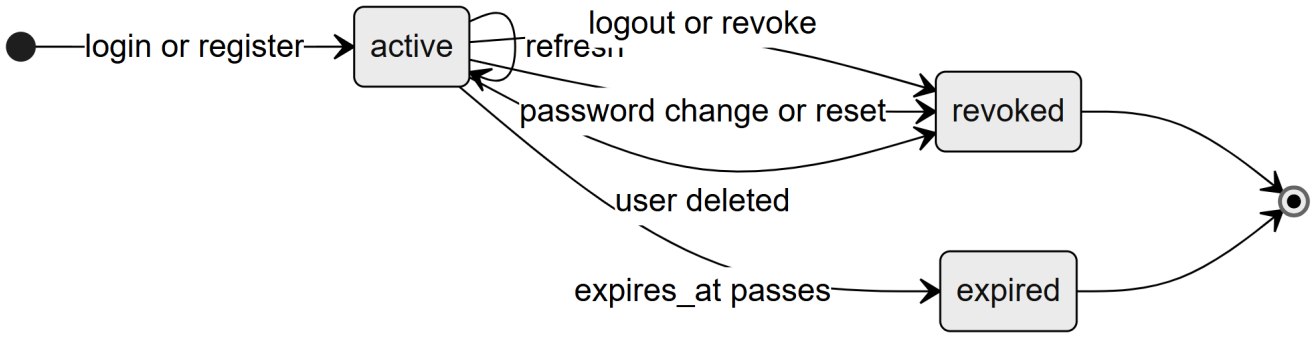


Figure 56.6 — Status lifecycle of a session.

The diagram shows a session starting at a dot and entering **active** when the user logs in or registers. A loop on active is labelled refresh: a refresh rotates the secret but keeps the same session and the same expiry. Active moves to **revoked** in three ways: the user logs out or revokes a session (single, all others or all), the user changes or resets the password, or the user is deleted. Active moves to **expired** when `expires_at` passes, which is one day or 30 days after sign-in. Revoked and expired are both final states. (A sixth sign-in also revokes the oldest session; the diagram leaves that case out to stay readable.)

What it does. Lets a user end the current session, see every active session with device details, and end any of them.

How the user does it. Open Settings, Sessions (or `/sessions`). Click **Revoke** on a row, **Sign out** on the current row, or **Sign out all others**. The toasts are "Session revoked", "Signed out successfully" and "All other sessions revoked".

Behind the scenes — technically.

Action	Endpoint	Notes
Sign out this session	`POST /auth/logout`	Public; reads the cookie, revokes that session, clears the cookie, writes a security log row
List active sessions	`GET /auth/sessions`	Newest activity first; marks the current one
Revoke one session	`DELETE /auth/sessions/:id`	Only the caller's own sessions; otherwise 404
Revoke all others	`POST /auth/sessions/revoke-all`	Keeps the current session
Sign out everywhere	`POST /auth/logout-all`	Revokes all sessions including the current one, clears the cookie

The table lists the five session endpoints. Revoking is a flag (`is_revoked`) on the row; rows are kept. There is no clean-up job for old session rows.

Rules and validations. A revoked session cannot refresh. An access token that was already issued stays valid until it expires, at most 15 minutes, because requests are checked by the token only.

Who can do it. Any signed-in user, on their own sessions.

Data read and written. `user_sessions`, `security_logs`.

Error cases. 404 "Session not found" for an id that is not the caller's.

Technical references: `backend/src/auth/auth.controller.ts` · `SessionService` · `web-app/src/features/auth/components/SessionManagement.tsx` · `web-app/src/routes/sessions.tsx`

56.3.7 Passwords: change, forgot and reset

Status: [Implemented] for change, forgot and reset. **Add password** is [Partial]: the endpoint exists but no screen calls it, because it is meant for accounts created through the disabled OAuth sign-in.

What it does. Lets a user change a known password, or recover from a forgotten one through an emailed link.

How the user does it.

- *Change:* Settings, Security, enter the current and new password. All other sessions are revoked, as the page says.
- *Forgot:* on the login page click "Forgot password?", enter the email and click "Send reset link". The page always says to check the inbox.
- *Reset:* open the link in the email, enter a new password.

Behind the scenes — technically.

- `POST /auth/change-password` (signed in) checks the current password with bcrypt, hashes the new one (12 rounds), and by default revokes every other session, keeping the current one.
- `POST /auth/forgot-password` (public, 3 per hour) always answers "If the email exists, a reset link will be sent", whether or not the address exists. For a real user it stores the SHA-256 hash of a random token, valid for 15 minutes, and emails the link.
- `PATCH /auth/reset-password` (public) hashes the supplied token, finds an unexpired match, sets the new password, clears the token and revokes **all** sessions of that user.
- `POST /auth/add-password` (signed in) sets a password only when the account has none; otherwise 400 "Account already has a password".
- All three paths write security log rows (change, reset requested, reset succeeded).

Rules and validations. Password rules are in section 56.5. A wrong current password gives 401 "Current password is incorrect". An invalid or expired reset token gives 400 "Invalid or expired token". Reset has no rate limit of its own; the token is single use and short lived.

Who can do it. Anyone for forgot and reset; signed-in users for change.

Data read and written. `users` (`password`, `password_reset_token`, `password_reset_expires`), `user_sessions`, `security_logs`.

Technical references: `AuthService.changePassword` · `AuthService.forgotPassword` · `AuthService.resetPassword` · `AuthService.addPassword` · `web-app/src/features/auth/components/ForgotPasswordForm.tsx` · `ResetPasswordForm.tsx` · `web-app/src/features/settings/pages/SettingsPage.tsx`

56.3.8 Rate limiting

Status: [Implemented], with a limit: the counters live in the memory of one API process.

What it does. Slows down guessing and abuse on the public and sensitive authentication routes.

Behind the scenes — technically. `RateLimiterGuard` counts requests per client address and route in a fixed window and answers 429 with `retryAfter` when the count is exceeded. The address comes from the first value of the forwarding header when present, otherwise the connection address.

Endpoint	Allowed	Window
`POST /auth/login`	5	15 minutes
`POST /auth/register`	5	1 hour
`POST /auth/verify-email`	10	15 minutes
`POST /auth/resend-verification`	3	1 hour
`POST /auth/forgot-password`	3	1 hour

The table lists the five throttled endpoints with their allowed attempts per window. Other routes, including refresh, reset-password and all clinical routes, have no limiter. A separate small throttle (120 requests per minute per address by default) protects the public lab magic-link routes (Chapter 41). Because the counters are in process memory, they reset when the process restarts and are not shared between API processes, so with several API processes the effective limit is higher. The code notes that a shared store such as Redis would fix this.

Who can do it. Not applicable; it applies to everyone.

Technical references: `backend/src/auth/guards/rate-limiter.guard.ts` · `RateLimiterGuard` · `RateLimit` · `backend/src/lab-access/magic-links/magic-link-throttle.guard.ts`

56.3.9 Security log

Status: [Partial]

What it does. Keeps a record of authentication events in the table `security_logs`: user, event type, IP address, user agent, session, success flag and a small metadata object.

What is written today. Successful sign-in, sign-out, sign-out everywhere, password change, password reset requested, password reset succeeded, and a failed sign-in only for suspended or deleted accounts. The event list in the code also names email verification, session, token, OAuth, account and suspicious-activity events, but nothing writes them yet. A wrong password does not produce a row.

What is missing. No endpoint or screen reads the security log, and no job deletes old rows, so it is a write-only record that can be queried in the database only.

Technical references: `backend/src/auth/security-log.service.ts` · `SecurityLogService` · `backend/src/auth/enum/SecurityEventType.ts` · `backend/src/auth/entities/SecurityLog.ts`

56.3.10 User administration and staff logins

Status: [Implemented]

What it does. Lets a super admin, an owner, or a user with the permission `admin.users.manage` create, edit, suspend and delete logins. The most common case is giving an existing employee a login.

How the user does it.

1. Open **User Admin** (`/users`) and click **Create User**.
2. Choose the type. For Staff, pick the employee from the list of staff who have no login yet.
3. Enter name, email and a password of at least 8 characters, and save.

Behind the scenes — technically.

Action	Endpoint
Profile of the caller	`GET /user/me`
List users by type (`super_admin` , `owner` , `employee`), paged	`GET /user`
Staff without a login	`GET /user/staff-candidates`
Create a user	`POST /user`
Edit a user (including status and password)	`PATCH /user/:id`
Delete a user	`DELETE /user/:id`

The table lists the user endpoints. The controller is context free and checks access itself: a super admin may act with or without an organization; anyone else must send an organization header and hold `admin.users.manage` at their location.

A **staff login** (type `employee`) runs in one transaction that locks the employee's active assignments in the organization. It refuses an employee who already has a login (409 "This employee already has a login") and an email already in use. It creates the user, then links the user to all of the employee's active assignments and switches portal access on for them. The login grants no permission by itself: what the person can do comes from the role of each assignment (Chapter 11). Changing assignments bumps the organization's access version, so the change reaches open tabs (Chapter 57).

Rules and validations.

- Only a super admin can create or edit a super admin. Only an owner can add, edit or delete another owner.
- A non-super-admin manager may touch only users of their own organization, never a super admin, and cannot elevate a user to super admin.
- Creating a user checks the plan's login limit for the organization (error code `PLAN_LIMIT`); super admins may exceed it.
- Delete is a soft delete: the status becomes `deleted` , a deletion time is set and every session of the user is revoked.
- Passwords created here need 8 characters, which is stricter than the 6 required elsewhere.

Who can do it. Super admin, owner, or `admin.users.manage` (high-risk; the Admin system role holds it).

Data read and written. `users` , `organization_ownership` , `employee_org_location_assignments` .

Error cases. 403 for the manager-scope rules in this list; 409 existing login; 400 duplicate email.

Technical references: `backend/src/user/user.controller.ts` · `UserService.createUser` · `createStaffLogin` · `staffCandidates` · `web-app/src/features/users/` · `web-app/src/routes/users.tsx`

56.3.11 Sign-in with Google or GitHub

Status: [Not implemented]

The code for Google and GitHub sign-in exists but is **disabled**: neither the controller nor the two strategies are registered in the authentication module, so the `/auth/google` and `/auth/github` routes do not exist. The user table has a `provider` field and an `oauth_providers` table, and the service has an OAuth login method, but none of it can be reached. To switch it on, a developer would register the three classes, read callback addresses from the environment and add the routes to the public route list. The Platform therefore supports email and password only.

Technical references: `backend/src/auth/oauth.controller.ts` · `backend/src/auth/auth.module.ts` · `AuthService.handleOAuthLogin`

56.3.12 The first super admin

Status: [Partial]

There is no screen, command or seed that creates a super admin. The usual path is to register an account and then set `users.global_role` to `super_admin` directly in the database (Chapter 7, section 7.5, gives the steps). After that, other super admins can be created from User Admin. A super admin signs in, chooses any active organization, and may open the setup pages (Organizations, Locations, Lab Directory, Config Studio, Subscriptions) even before choosing a location.

Technical references: `backend/src/common/enums/global-role.enum.ts` · `web-app/src/routes/__root.tsx` · `SUPER_ADMIN_SETUP_PATHS`

56.4 Entities

The tables involved are `users`, `user_sessions`, `security_logs` and the unused `oauth_providers`; Chapter 13 describes them column by column. In short: each session belongs to exactly one user and is deleted if the user row is deleted; a security log row points to a user but is kept (the link is set to null) if the user is removed; a user is linked to organizations as an owner through the ownership table and to employees through assignments.

56.5 Password rules

Status: [Implemented]

Where	Minimum length
Register, login, change password, reset password	6 characters
Create user in User Admin	8 characters

The table shows the lengths the API enforces. There is no rule about letters, digits or symbols, no list of banned passwords and no expiry. The sign-up and reset pages show a strength meter, but it only advises. Passwords are stored as bcrypt hashes with 12 rounds and are never returned by the API.

56.6 Limitations and known gaps

- Revoking a session does not cancel an access token that was already issued; it works until it expires, at most 15 minutes.
- Rate-limit counters are per process and lost on restart.
- The security log has no reader and no clean-up; failed password attempts are not recorded.
- No two-factor authentication, no password strength enforcement and no lockout of an account after repeated failures.
- Session rows are never deleted, only flagged.

56.7 Owner's FAQ

How long does a sign-in last? One day by default and 30 days when "Keep me signed in for 30 days" is ticked. The length is fixed at sign-in; using the application does not extend it.

If an employee leaves, how do I cut off access at once? Set the user's status to suspended or delete the user in User Admin. Deleting revokes every session; a suspended user is refused at the next refresh. An access token already issued may still work for up to 15 minutes.

Can one person have several logins on different devices? Yes, up to five active sessions. A sixth sign-in revokes the oldest.

What happens if someone forgets their password? They use "Forgot password?". The link lasts 15 minutes and a successful reset signs the user out of every device.

Can staff sign in with Google? No. That feature is built but disabled.

Where do I see failed sign-in attempts? Nowhere in the application today. Only the throttle protects the login route.

Chapter 57 — Access control in depth

This chapter follows the Platform's access control from the first guard that sees a request to the last button hidden in the browser. It is written for the product owner, who needs to know where a permission is checked, how quickly a change reaches users and which protections do not exist. Chapter 11 explains the permission catalog, the system roles and the roles screen; this chapter does not repeat them and goes deeper into the mechanics. Customization screens (features, roles, sidebar, theme) are in Chapter 58.

57.1 The idea in one minute

Status: [Implemented]

Access control answers four separate questions, in this order: Is this a real, signed-in user? Is this user allowed to work in this organization and location? Is the feature switched on for this clinic? Does the user's role hold the permission this action needs? A fifth check asks whether the clinic's subscription still allows writing. Each question is a global guard that runs before the controller. After the guards, two more mechanisms narrow what a person sees inside a permitted action: **row scope** (only own, location or organization rows) and **field hiding** (prices removed from a response). The browser repeats the same questions only to decide what to show; the server never trusts the browser.

Technical references: `backend/src/app.module.ts` · `backend/src/access/` · `web-app/src/features/access/`

57.2 The five guards in order

Status: [Implemented]

Five global guards run on every HTTP request, in a fixed order, and the first refusal stops the request (Chapter 5, section 5.4, lists them with their skip rules).

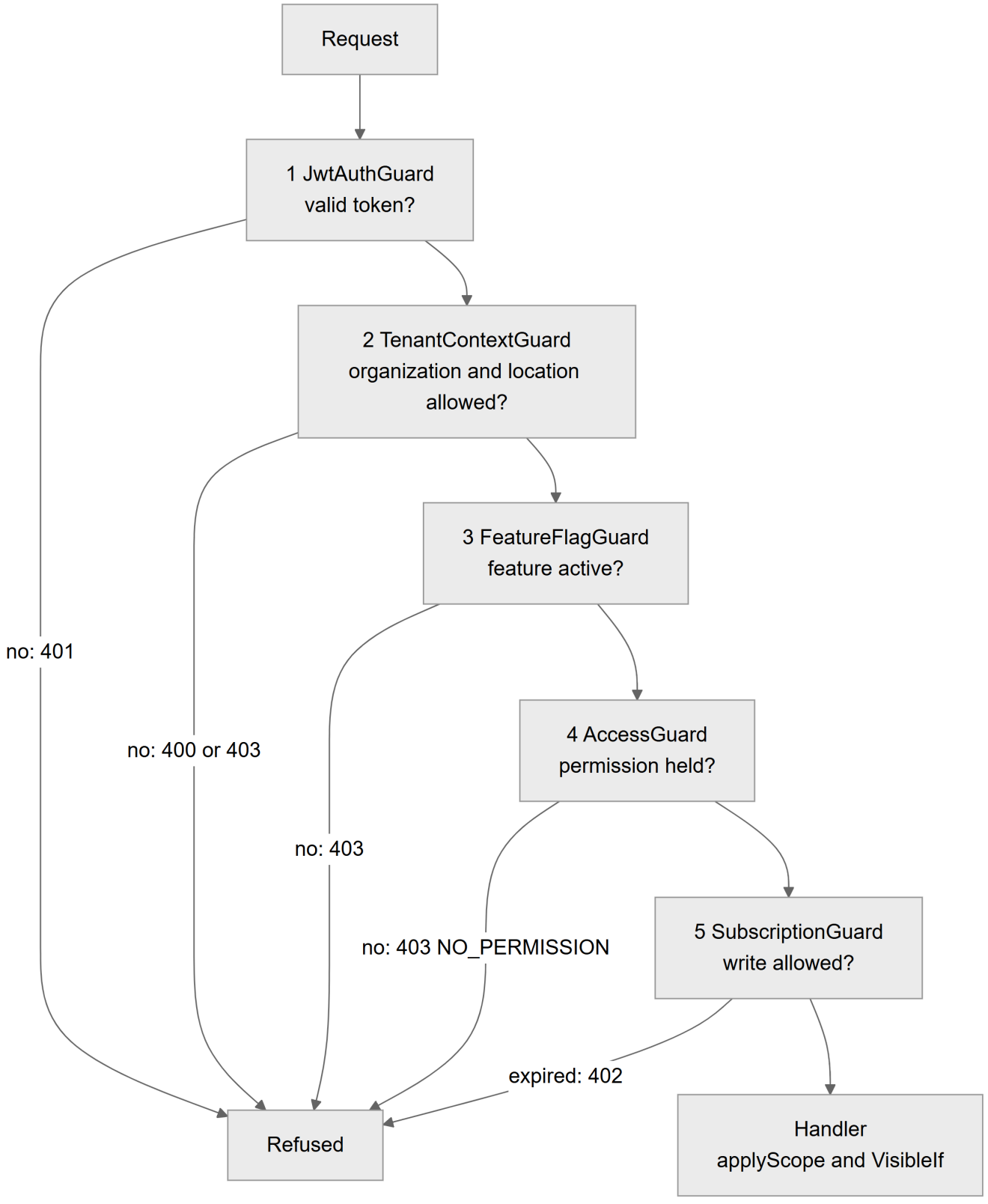


Figure 57.1 — The five global guards in the order they run, with the refusal each can give.

The flowchart reads from top to bottom. A Request enters guard 1, `JwtAuthGuard`, which asks whether the token is valid; if not, the answer is 401 and the request is refused. Guard 2, `TenantContextGuard`, asks whether the organization and location in the headers are allowed for this user; if not, the answer is 400 (headers missing) or 403. Guard 3, `FeatureFlagGuard`, asks whether the required feature is active; if not, 403. Guard 4, `AccessGuard`, asks whether the permission is held; if not, 403 with the code `NO_PERMISSION`. Guard 5, `SubscriptionGuard`, asks whether a write is allowed; if the subscription has expired the answer is 402. A request that passes all five reaches the handler, where row scope and field hiding apply. Every refusal leads to the same "Refused" box.

The details that matter to an owner:

- **Guard 1 and public routes.** `JwtAuthGuard` skips only the routes in a fixed public list (sign-in, register, refresh, password routes, email verification, logout, the public theme, the waiting-room display state and the lab magic links). Marking a handler `@Public()` is not enough on its own; the route must also be in that list.
- **Guard 2 and super admins.** A super admin passes without headers. `@RequireSuperAdmin()` routes are checked here: anyone else gets 403 "Super admin access required". `@ContextFree()` routes skip the tenant check, and the handler then checks access itself. The guard has no path-prefix exceptions. A user passes for an organization if they own it, or if they have an active assignment at that exact location with portal access on.
- **Guard 3 only runs for tagged routes.** `@RequireFeature('key')` is needed only on routes that use `@AnyMember()`, because a permission already implies its feature (the access context drops permissions of inactive features).
- **Guard 4 attaches the access context.** Whenever the request names an organization, it puts the resolved access context on the request for the handler, even on routes with no permission marker.
- **Guard 5 is read-only mode.** It refuses writes with 402 `SUBSCRIPTION_EXPIRED` for a clinic whose subscription has lapsed. Reads, super admins and the paths for signing in, the account, `/me/` and subscription administration stay open.

Technical references: `backend/src/auth/guards/jwt.guard.ts` · `backend/src/auth/guards/tenant-context.guard.ts` · `backend/src/feature-flags/guards/feature-flag.guard.ts` · `backend/src/access/access.guard.ts` · `backend/src/subscriptions/subscription.guard.ts`

57.3 The decorators

Status: [Implemented]

Decorator	Where	What it means	Enforced by
`@Can('a', 'b')`	handler or controller	Every listed permission is required; class-level and handler-level lists are merged	`AccessGuard`
`@AnyMember()`	handler or controller	Any active member of the location (owners and super admins too)	`AccessGuard`
`@Public()`	handler or controller	No sign-in required (plus the public route list)	all guards skip
`@ContextFree()`	handler or controller	No tenant headers needed; the handler checks access itself	`TenantContextGuard` skips
`@RequireSuperAdmin()`	handler or controller	Platform operator only	`TenantContextGuard`
`@RequireFeature('key')`	handler or controller	The feature must be active; class and handler lists are merged	`FeatureFlagGuard`
`@Access()`	handler parameter	Injects the `AccessContext` of the request	decorator
`@VisibleIf('perm')`	property of a response class	The field is removed from the response unless the caller holds the permission	global interceptor

The table lists the eight decorators with their placement and meaning. The first five say who may call a handler. `@RequireFeature` adds a feature condition. `@Access()` and `@VisibleIf` are tools for handlers and response classes.

The coverage test. The test `access-coverage.spec.ts` loads every controller and fails if any handler lacks one of the markers `@Can`, `@AnyMember`, `@Public`, `@RequireSuperAdmin`, `@ContextFree` or an explicit `@UseGuards(...)`, or if any handler still uses the removed role decorator. Controllers under the path prefixes for authentication, sessions and health, and the hello endpoint, are exempt. This test is the safety net, because at runtime a handler with no marker passes the access guard.

Technical references: `backend/src/access/decorators/access.decorators.ts` · `backend/src/common/decorators/` · `backend/src/access/access-coverage.spec.ts`

57.4 The access context

Status: [Implemented]

For each request that names an organization, `AccessResolver` builds an `AccessContext`: user id, organization, location, access version, super-admin flag, owner flag, member flag, employee id, active features, granted permissions (feature-filtered, with implied permissions expanded), the role and its home page. Owners and super admins receive every permission at its widest scope. Staff receive the permissions of the role on their assignment at the active location, and only if portal access is on for that assignment. A role that is inactive or belongs to another organization gives nothing. A missing key means no access.

The context offers a small set of questions that code uses instead of role names:

- `can(permission)` and `canAll([...])`: is it granted.
- `scope(permission)`: `own`, `location` or `org`, or nothing. An unscoped grant reads as `org`.
- `applyScope(queryBuilder, permission, columns)`: adds the row filter to a database query. With `own` it adds a condition on the caller's employee id (or a custom condition). With `location` it adds a condition on the active location. With `org` it adds nothing, because the organization filter is the caller's. With no grant, or when the caller has no employee record or no location to match, it adds a condition that matches no rows. If a developer forgets to name the column, it throws an error rather than silently showing nothing.
- `decide(permission, { value, min, max, approvalFlow })`: returns allow, deny or "needs approval". Inside the range the answer is allow; outside it, allow only when the caller holds the escalation permission (today: preset discounts are escalated by the custom-discount permission), otherwise needs approval when the clinic runs an approval flow, else deny.

Field hiding. A response class can mark a property `@VisibleIf('patients.price.view')`. A global interceptor removes that property from every instance in the response when the caller lacks the permission, and removes every guarded field when there is no access context. It works only on class instances, not on plain object literals.

Technical references: `backend/src/access/access.resolver.ts` · `backend/src/access/access-context.ts` · `AccessContext` · `backend/src/access/visible-if/visible-if.ts` · `backend/src/patients/access/patient-scope.ts`

57.5 What the web app receives: `GET /me/access`

Status: [Implemented]

`GET /me/access` is a context-free route that checks membership itself. It needs the organization header and, for staff, the location header; without a location, staff roles give no permissions because roles are per location. It answers 400 without an organization, 403 when the caller has no access there. The answer has these parts: the access version; the organization and location ids; the super-admin and owner flags; the list of active feature keys;

the granted permissions with their scope (or `true`); the roles (id, name and template key); the employee id; the home page; and the clinic's sidebar layout, which is null while the clinic uses the default. `GET /access/catalog` (needs `admin.roles.manage`) returns the catalog, bundles and role templates for the roles screen.

Technical references: `backend/src/access/access.controller.ts` · `MeAccessController` · `AccessCatalogController`

57.6 In the browser

Status: [Implemented]

- **`AccessProvider`** loads `/me/access` for the active organization and location. The data is fresh for 30 seconds and refetched when the window regains focus, when the realtime socket announces `access.changed` and after a reconnect.
- **`useAccess()`** returns `can(permission)`, which takes one permission or a list and means any-of; `canAll(list)` for all-of; `scope(permission)`; feature checks; the owner and super-admin flags; the home page and the sidebar layout. The `<Can perm="...">` component wraps content the same way. When no payload exists yet for a super admin on a setup page, a fallback allows everything; a loaded payload is always the source of truth.
- **Route gate.** Each route file declares `staticData: { can: ... }`. One component, `RouteAccessGate`, mounted in the root route (`__root.tsx`), reads the requirement of every matched route and shows the No access page when any requirement fails. While access is loading it shows a skeleton; if loading fails it shows "Couldn't load your access" with a retry button. Routes without a requirement pass.
- **Sidebar.** The sidebar comes from a developer-owned registry of 39 items. Each item declares its own requirement: a permission (any-of), `superAdminOnly` (six items: Organizations, Subscriptions, Lab Directory, Form schemas, Workflows, Stage templates), `labMember` (the Lab portal item) and `ownerOrSuperAdmin` (the flag exists but no item uses it today). The clinic's saved layout only arranges, renames and hides these items; items a user may not open are removed before display. Registry items that the layout forgot appear in a trailing "More" section so a new page never disappears silently.
- **Home page.** After sign-in, the role's home page is used, then the clinic's home page, and the page is opened only if the user passes the same sidebar and route checks; otherwise the Dashboard opens.

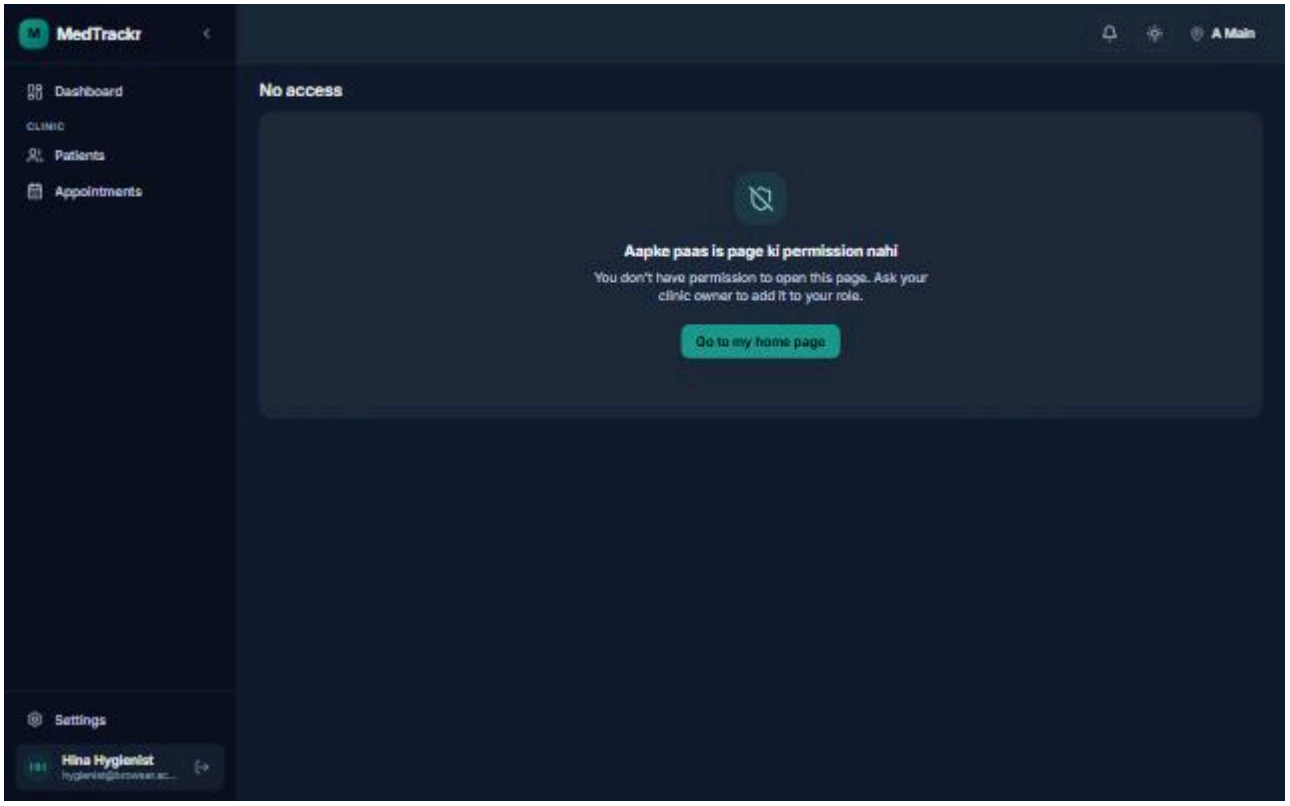


Figure 57.2 — The No access page shown when a role lacks the permission of the page.

The screenshot was taken on 2026-10-23 in dark theme and shows the former working name of the product in the corner; the product is now called the Platform and each clinic sets its own name. A test user in a custom role opened a page their role does not allow. The sidebar is short: Dashboard, and under "Clinic" only Patients and Appointments, with Settings and the user at the bottom. The main area is titled "No access" and shows a shield icon, a Roman-Urdu headline meaning that the user has no permission for this page, an English sentence saying the user does not have permission and should ask the clinic owner to add the page to their role, and a button, "Go to my home page". The user can click the button to go to the role's home page, or to the Dashboard if the role has none.

Technical references: `web-app/src/features/access/components/AccessProvider.tsx` · `RouteAccessGate.tsx` · `NoAccess.tsx` · `web-app/src/features/access/lib/access.ts` · `web-app/src/features/access/nav/nav.registry.ts` · `resolveNav.ts` · `homeRoute.ts` · `web-app/src/routes/__root.tsx`

57.7 How fast a change reaches users

Status: [Implemented]

Each organization has a counter, `organizations.access_version`. It is raised by one after any committed change that can alter someone's access: employment roles (any column), assignments (role, location, organization, active flag, portal access or linked login), organization feature flags, ownership, location feature overrides and sidebar layout changes. An entity subscriber does this automatically for writes made through the data layer, once per transaction after the commit and never on rollback. Code that uses raw SQL or database cascades must bump explicitly; the employee delete path does.

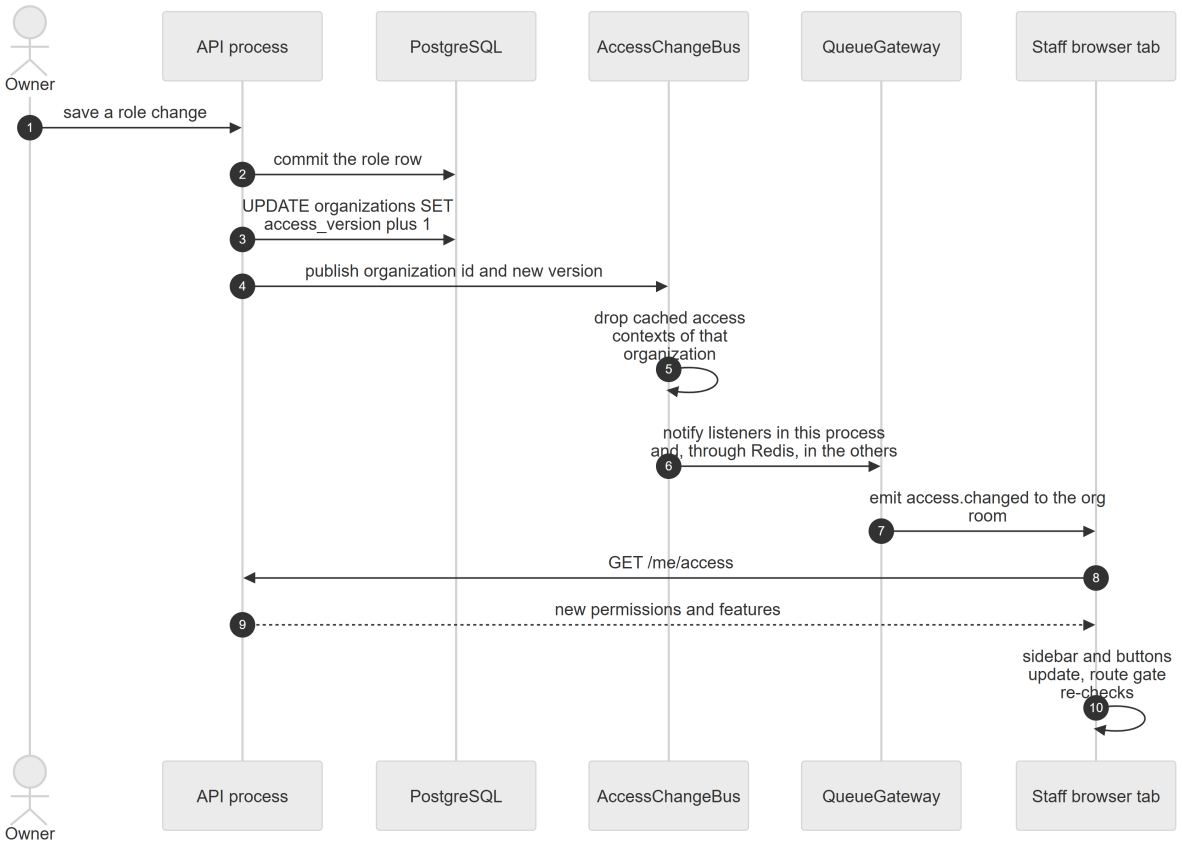


Figure 57.3 — Sequence from an owner's role change to the refreshed browser tabs.

The diagram has six participants: the Owner, the API process, PostgreSQL, the `AccessChangeBus`, the `QueueGateway` and a Staff browser tab. (1) The owner saves a role change. (2) The API commits the role row. (3) The API increments `access_version` of the organization. (4) The API publishes the organization id and new version to the bus. (5) The bus drops cached access contexts of that organization. (6) It notifies listeners in this process and, through Redis, in the other processes. (7) The gateway emits `access.changed` to the organization's room. (8) The browser tab calls `GET /me/access`. (9) The API answers with new permissions and features. (10) The sidebar and buttons update and the route gate checks again.

The timing rules are:

- **Version cache.** Each process caches the current version of an organization for 15 seconds. The bus updates it at once in every process that hears the change.
- **Context cache.** Access contexts are cached in process memory, at most 10,000, for at most 5 minutes, keyed by user, organization, location and version. A new version means a new key, so stale entries are never used.
- **Redis.** One Redis channel carries access-change messages between API and worker processes. If Redis is down, only the process that made the change learns at once; the others catch up within 15 seconds, when their cached version expires.
- **Feature cache.** The enabled-feature set is cached for a short time and dropped by the same bus.
- **Browser.** Open tabs refetch when told, on focus, and in any case after 30 seconds of staleness.

In practice, an owner's change reaches other staff within seconds, and within 15 seconds even when Redis is unavailable. A person who is already inside a permitted request is not interrupted.

Technical references: `backend/src/access/version/access-version.service.ts` · `AccessVersionService` · `access-version.subscriber.ts` · `access-change-bus.ts` · `web-app/src/features/access/hooks/useAccessSocket.ts`

57.8 The lab portal is a separate door

Status: [Implemented]

Lab staff do not use the clinic guards for their screens. Every lab portal controller uses its own `LabPortalGuard` after the global guards (which skip these routes as context free). It rejects any request carrying clinic headers (400 `LAB_PORTAL_CLINIC_CONTEXT`), so lab and clinic scopes cannot mix. It requires an active membership of an active lab; the lab is named by an `x-lab-id` header, or is the user's only lab (several labs without a header gives 400 `LAB_SELECTION_REQUIRED`). Another lab's id looks exactly like a lab that does not exist (403 `LAB_ACCESS_DENIED`). A lab role list on the handler (lab admin or technician) is checked against the membership. Super admins hold no lab membership and are refused. Chapter 41 describes the portal itself.

Technical references: `backend/src/labs/portal/lab-portal.guard.ts` · `LabPortalGuard` · `backend/src/labs/portal/lab-memberships.ts`

57.9 Row-level security does not exist

Status: [Not implemented]

The database has no row-level security policies. Tenant isolation is enforced by the application: handlers filter every query by the organization (and often the location) taken from the validated tenant context, and by `applyScope` for own, location and organization reach. A bug in a query that forgets the organization filter would therefore not be stopped by the database. The safeguards are the coverage test, the single `@ActiveTenant()` source of the organization, and code review. Chapter 8 describes the tenant model.

57.10 Limitations and known gaps

- Tenant isolation is application-level only (section 57.9).
- A handler without a marker passes the access guard at runtime; only the coverage test prevents it.
- Contexts and version numbers are cached in process memory; with Redis down, other processes may lag up to 15 seconds.
- An access token already issued is not re-checked against the session; permissions, however, are checked on every request from the access context, so a role change takes effect without a new token.
- The `ownerOrSuperAdmin` sidebar flag is unused today.

57.11 Owner's FAQ

I removed a permission from a role. When does it stop working? Within seconds for open screens (the browser is told over the realtime connection) and on the next request on the server. At worst 15 seconds if the message bus is down.

A user opens a page by typing its address. Can they bypass the sidebar? No. The route gate shows No access and the API refuses the data calls with 403 `NO_PERMISSION`. Hiding a sidebar item is cosmetic; the checks are the permission on the route and on the API.

Does the sidebar layout give anyone access? Never. The layout only arranges pages the registry knows; every item still needs the viewer's permission.

Can the database stop one clinic seeing another's data? Not by itself. The application filters every query by organization; there are no database row-level policies.

Who can see which patients? The scope of `patients.view`: a doctor sees own patients, front desk sees the location, an admin sees the organization (Chapter 11).

What does 402 mean? The clinic's subscription expired; the clinic is read-only until a payment is recorded (Chapter 45).

Chapter 58 — Clinic customization

This chapter describes how a clinic shapes the Platform to its own way of working: which features are on, which roles exist and what they may do, how the sidebar is arranged, how the application looks, and where each role starts. It is written for the product owner, who needs to know what an owner can change without a developer and what only the platform team can change. Four setup screens carry this work: Features, Roles and permissions, Sidebar and Branding and theme. The permission model behind them is in Chapters 10 and 11 and the access mechanics are in Chapter 57; this chapter restates what is needed and describes the screens.

58.1 The four setup screens

Status: [Implemented]

Screen	Route	Sidebar label (section "Clinic setup")	Permission to open
Features	`/admin/features`	Features	`admin.features.manage`
Roles	`/admin/roles`	Roles & permissions	`admin.roles.manage`
Sidebar builder	`/admin/navigation`	Sidebar	`admin.nav.manage`
Branding and theme	`/admin/theme`	Theme	`admin.theme.manage`

The table lists the four screens, their routes, the labels in the sidebar and the permission each route needs. The owner always has all four permissions. The Admin system role holds them by default. Every save on these screens increases the organization's access version or invalidates the settings cache, so changes reach other users quickly (Chapter 57, section 57.7).

Technical references: `web-app/src/routes/admin.features.tsx` · `admin.roles.index.tsx` · `admin.navigation.tsx` · `admin.theme.tsx` · `web-app/src/features/access/nav/nav.registry.ts`

58.2 Features screen

Status: [Implemented]

What it does. Lets the owner switch the features in the clinic's plan on or off for the whole clinic, or switch one off at a single location. A feature is only available if the platform team has **entitled** the clinic to it; the owner then decides whether it is **enabled**; and a **location override** can switch it off at one branch. Chapter 10 describes the three switches and the catalog in full.

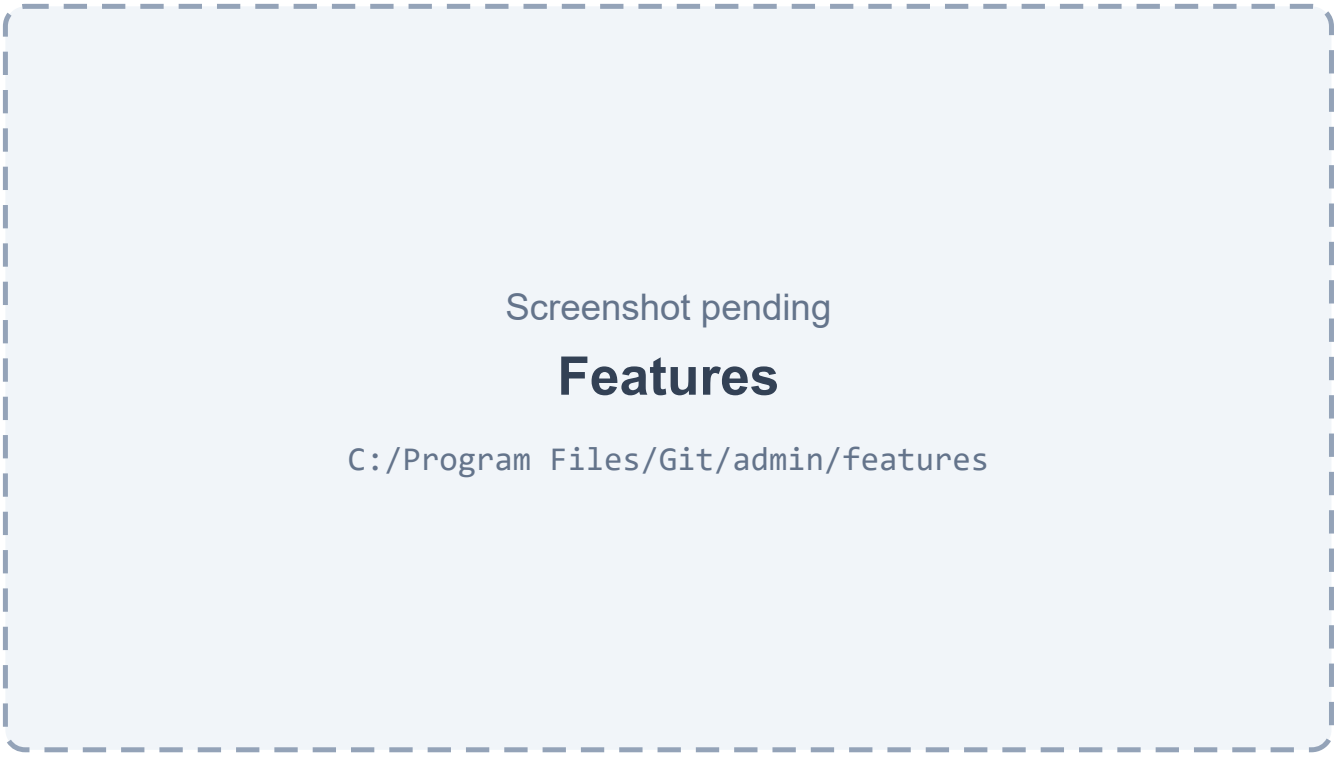


Figure 58.1 — Features screen (screenshot pending).

The page is titled "Features" with the subtitle "Turn the features in your plan on or off for the clinic, or for one location." Features are listed in rows, each with a name, a switch and a button that opens the locations sheet. A filter row narrows the list. Features the clinic is not entitled to cannot be switched on. If there are no features the page says "No features yet" and that the platform has not set up any features for the clinic.

How the user does it.

1. Open **Features** in the sidebar.
2. Move a switch to turn a feature on or off. Turning a feature off first opens a dialog titled "Turn off ?" that lists the permissions the feature carries and the roles that will lose them. This is the **impact preview**.
3. Click **Turn off** to confirm.
4. To switch a feature off for one branch only, open the locations sheet and move the switch of that location.

Behind the scenes — technically.

Action	Endpoint
List features for the clinic	`GET /admin/features`
Impact of turning one off	`GET /admin/features/:key/impact`
Switch on or off	`PATCH /admin/features/:key`
Switch off or on at one location	`PUT /admin/features/:key/locations/:locationId`

The table lists the four endpoints; all need `admin.features.manage`. The impact call returns the catalog permissions the feature carries and, for each active role of the organization that grants any of them, the permissions it would lose. Turning on a feature the clinic is not entitled to gives 403 `NOT_ENTITLED`. Every change writes an `audit_logs` row; a super admin acting inside a clinic is recorded as the platform.

Technical references: `backend/src/feature-flags/admin-features.controller.ts` · `FeatureAdminService.getImpact` · `web-app/src/features/feature-settings/`

58.3 Roles builder

Status: [Implemented]

What it does. Lets the owner (or anyone with `admin.roles.manage`) create custom roles, copy a system role, choose a level for each module, set a home page and see members. Chapter 11, section 11.5, describes the rules in detail: escalation protection, reason for high-risk grants, read-only system roles, unique names and the ban on deleting a role that has members.

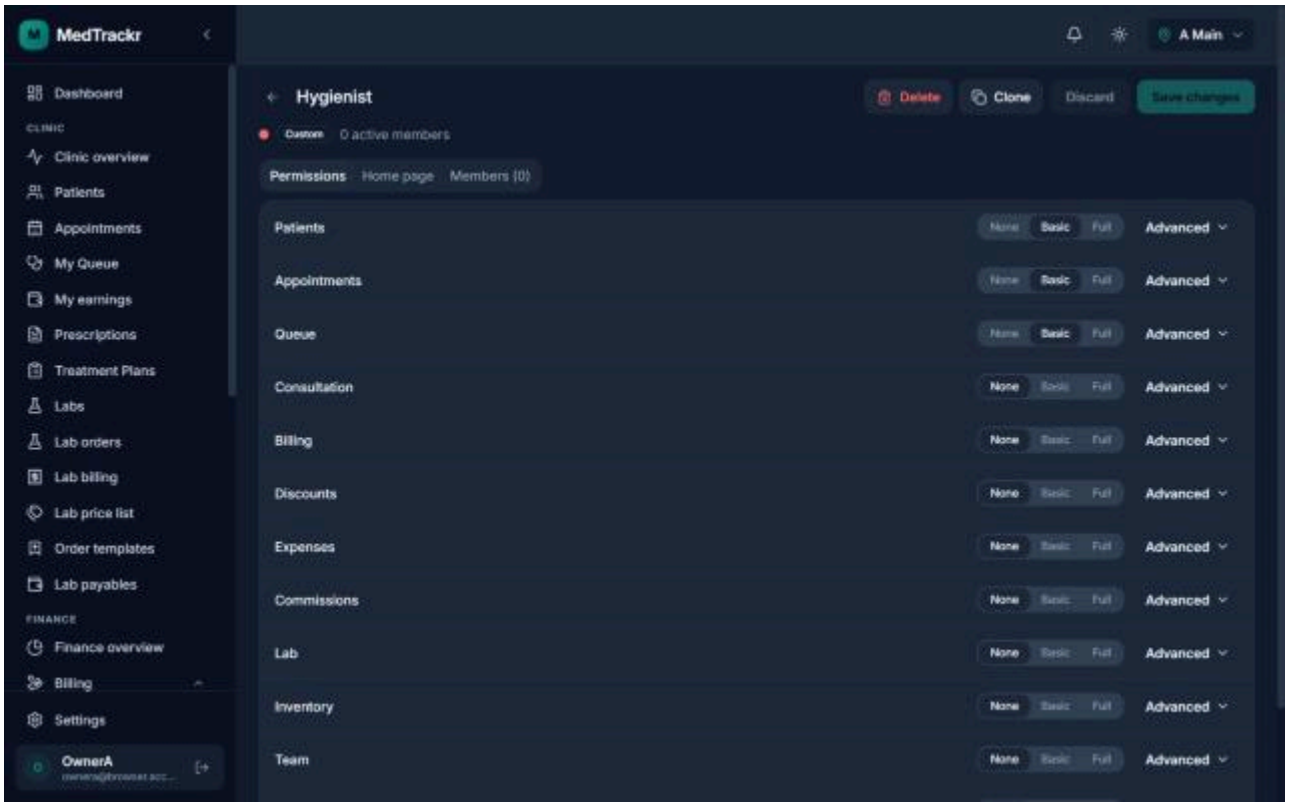


Figure 58.2 — The role editor for a custom role, showing None, Basic and Full per module.

The screenshot was taken on 2026-10-23 in dark theme and shows the former working name of the product in the corner. It shows a custom role open in the editor. The header holds the role name and the actions Delete, Clone, Discard and Save changes. Three tabs sit under it: Permissions, Home page and Members. On the Permissions tab each module has a row with three level buttons, None, Basic and Full, and an Advanced control that opens the module's full permission list with a check box and, for scoped permissions, a scope selector. A module whose permissions match no level exactly shows Custom. The left sidebar shows the full owner menu.

What the levels mean. None grants nothing from the module. **Basic** grants an everyday set. **Full** grants every permission of the module at its widest scope. **Advanced** lets the owner tick individual permissions. **Clone** copies any role, including a system role, with the same permissions and home page, and moves nobody to the copy. **Members** lists the employees who hold the role, per location; roles are assigned on each employee's page.

Technical references: `backend/src/employees/employment-roles.admin.controller.ts` · `backend/src/access/catalog/bundles.ts` · `web-app/src/features/roles/`

58.4 Per-role home page

Status: [Implemented]

What it does. Sets the page a member of the role lands on after signing in. The five system roles have defaults (Admin: dashboard; Doctor: the doctor's queue; Receptionist and Assistant: appointments; Accountant: billing).

How the user does it. Open a custom role, choose the **Home page** tab, pick a page in the "Page" list and save. The clinic as a whole can also set a **Clinic home page** on the Sidebar screen; it is used when the role sets none.

Rules and validations. The value must be an in-app path starting with a single slash, with no address or scheme. At sign-in the web app uses the role's page, then the clinic's page, then the Dashboard; the page opens only if the user passes the permission checks for it, otherwise the Dashboard opens. A home page never widens access.

Technical references: `web-app/src/features/roles/components/HomePageTab.tsx` · `web-app/src/features/access/nav/homeRoute.ts` · `backend/src/access/nav/nav-layout.dto.ts`

58.5 Sidebar builder

Status: [Implemented]

What it does. Lets the owner rename, hide, group and reorder the pages in the sidebar for the whole clinic. Each person still sees only the pages they may open, so the layout can never give access.



Figure 58.3 — Sidebar builder (screenshot pending).

The page is titled "Sidebar" with the subtitle "Rename, hide and reorder the pages in your clinic's sidebar. Each person still sees only the pages they have access to." A line at the top says whether the clinic uses the standard sidebar or a custom one, or that there are unsaved changes. The buttons are **Discard changes** (when dirty), **Reset to default** and **Save**. A "Clinic home page" selector (default: Dashboard) comes first. The editor is a drag-and-drop area of sections; each section has a label that can be renamed, can contain pages and collapsible groups, and has a remove button. An **Add section** button adds a section. Pages that are not in any section wait in a tray called "Not placed". Each page can be renamed inline, hidden with a toggle, or dragged to another place. On the right, a live preview shows the sidebar as the owner would see it, with a note that other roles see only their pages. Leaving with unsaved changes asks "Leave without saving?".

How the user does it.

1. Open **Sidebar** under Clinic setup.
2. Drag pages between sections or the tray, rename sections and pages, hide pages, add groups.

3. Choose a clinic home page.
4. Click **Save**. To undo everything, click **Reset to default** and confirm; sections, renames, hidden pages and the clinic home page are removed.

Behind the scenes — technically. The layout is stored as one JSON document per organization in `nav_layouts`; a column for a role exists but only the clinic default is written today. Endpoints (all need `admin.nav.manage`): `GET /admin/navigation` (null while the clinic uses the default), `PUT /admin/navigation` and `DELETE /admin/navigation`. Saving and resetting bump the access version and write an audit row.

Rules and validations.

- At most 20 sections; labels at most 40 characters; at most 100 items in a list.
- Item keys must look like `nav.something`; no key may appear twice anywhere.
- A section entry is either a link (a key) or a group (a label and items, no key).
- The home route must be an in-app path.
- Footer items (Settings) stay pinned and cannot be arranged.
- Pages the layout forgets appear in a "More" section, so nothing disappears by accident.

Data read and written. `nav_layouts`, `audit_logs`.

Technical references: `backend/src/access/nav/nav-layouts.controller.ts` · `NavLayoutsService` · `nav-layout.dto.ts` · `web-app/src/features/nav-builder/` · `web-app/src/features/access/nav/resolveNav.ts`

58.6 Sidebar registry flags

Status: [Implemented]

The list of pages that can appear is owned by developers in `nav.registry.ts`: 39 items. The clinic's layout only arranges them. Each item has one requirement: a permission or list of permissions (any-of), `superAdminOnly` (Organizations, Subscriptions, Lab Directory, Form schemas, Workflows and Stage templates), `labMember` (Lab portal) or `ownerOrSuperAdmin` (defined but unused). Section 57.6 describes how the sidebar is filtered. Adding a new page needs a code change; the owner cannot create sidebar items.

58.7 Theme and branding

Status: [Implemented]

What it does. Controls the clinic's name, logos, brand colour, sidebar style and default light or dark mode, for staff screens, the login page and printouts.

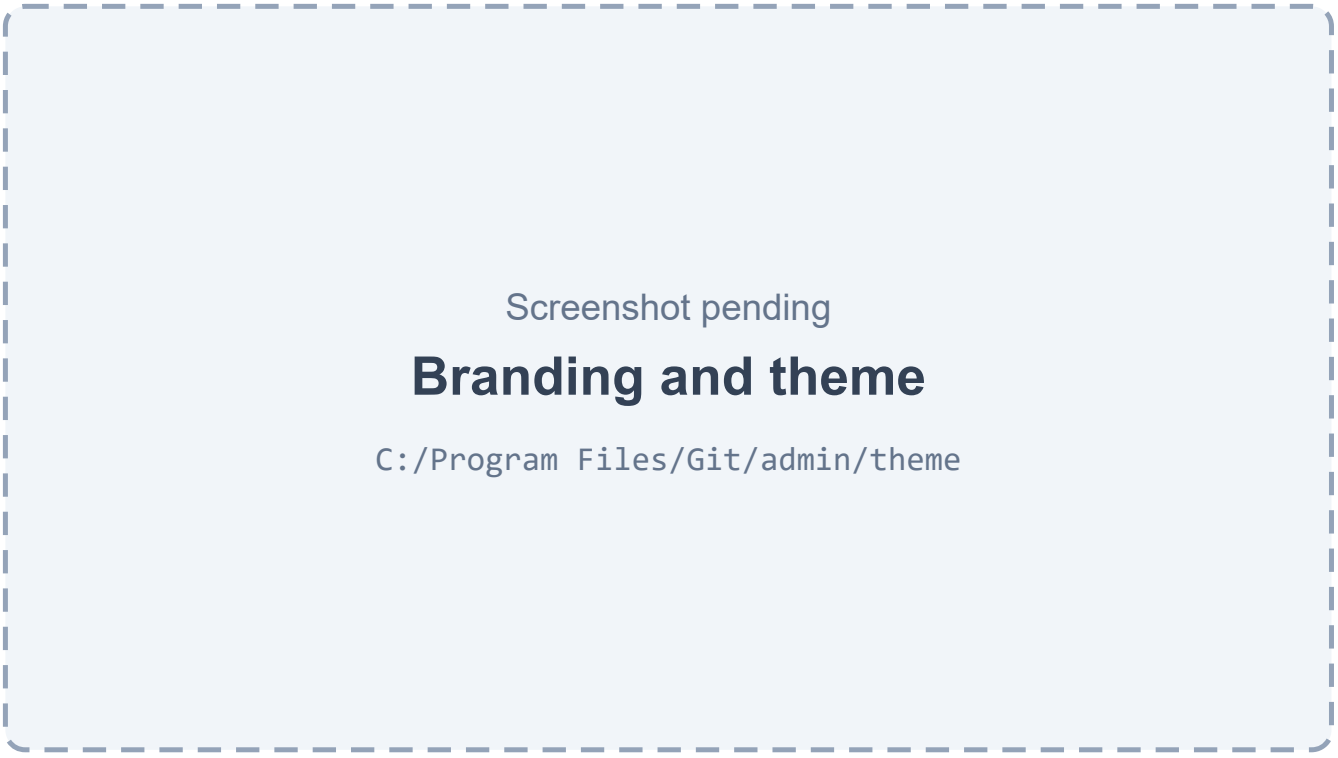


Figure 58.4 — Branding and theme (screenshot pending).

The page is titled "Branding & theme" with the subtitle "Your clinic's name, logo, colours and look — for staff screens, the login page and printouts." It has a back link to Settings and a button that opens the per-location sheet. The controls are on the left and a live preview is on the right. The controls are: Clinic name; Logo (light); Logo (dark); Browser tab icon; Brand colour; Sidebar style; Default mode. Each can carry a lock switch labelled "Locations can't override". Buttons are **Save theme** (enabled only when the form changed and is valid) and a reset button that asks "Reset to default?".

The seven keys.

Key	Meaning	Allowed values	Scopes that may set it
`theme.app_name`	Clinic name shown in the application (default "Clinic Portal")	1 to 60 characters	organization, location
`theme.logo_url`	Light-mode logo	http or https address	organization, location
`theme.logo_dark_url`	Dark-mode logo	http or https address	organization, location
`theme.favicon_url`	Browser tab icon	http or https address	organization
`theme.brand_color`	Brand colour (default blue)	`#rrggbb`	organization
`theme.sidebar_style`	Sidebar look	light, dark or brand	organization
`theme.default_mode`	Starting mode	light, dark or system	organization

The table lists the seven theme keys with their meaning, allowed values and the levels that may set them. Only the name and the two logos can differ per location. The platform level may set any key. Each key can be locked by the platform for organizations, or by the organization for its locations.

The cascade.

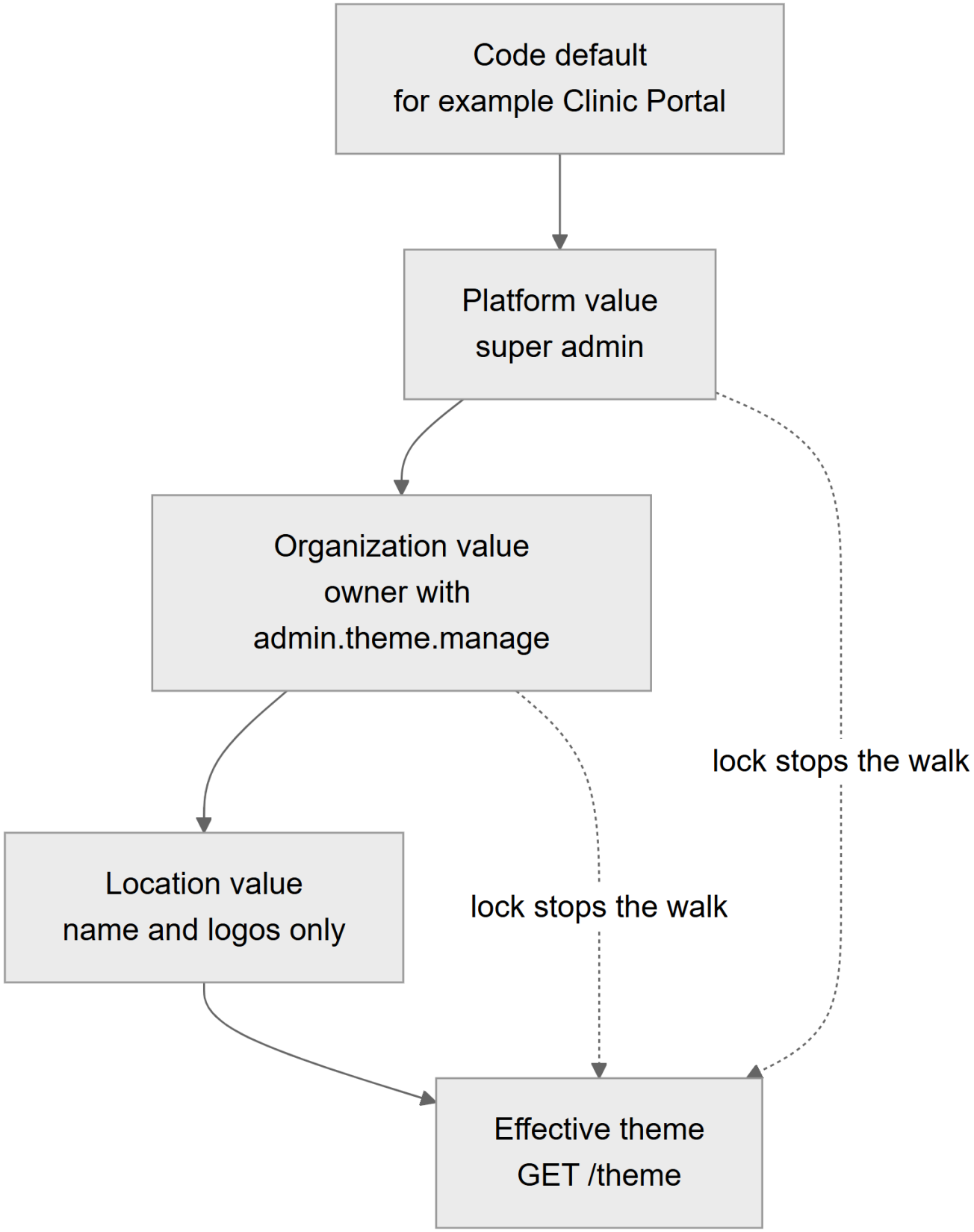


Figure 58.5 — The theme cascade from code default to the effective theme, with locks.

The flowchart reads from top to bottom. The code default (for example the name "Clinic Portal") feeds the Platform value set by the super admin. That feeds the Organization value set by an owner with `admin.theme.manage`. That feeds the Location value, which may set only the name and logos. The last box is the

effective theme returned by `GET /theme`. Two dotted arrows from the Platform and Organization boxes straight to the effective theme show that a lock stops the walk: a value locked at a higher level wins over anything set lower down.

Logos. A logo is uploaded with `POST /uploads`, which accepts one file up to 15 MB; the theme screen accepts images up to 2 MB and stores the returned address as the value.

Brand colour contrast check. When the owner picks a colour, the screen checks it against accessibility rules. It warns if the button text on the colour has less than 4.5 to 1 contrast, and warns if the colour on a white page has less than 3 to 1 (links and outlines will be hard to see, and a darker shade is used for brand text). When both pass, the screen says "Contrast OK" with the ratio. The warning is advice; it does not block saving.

Public theme for the login page. `GET /public/theme/:organizationId` (optionally with a location) is public and returns theme tokens only, nothing else about the clinic. An unknown organization gets the platform theme. The login page learns which clinic to show from an organization parameter in the address or from the last clinic used in that browser. The browser also keeps a copy of the last theme and its colours so the loading screen is on-brand before the server answers; local storage is used only for this convenience.

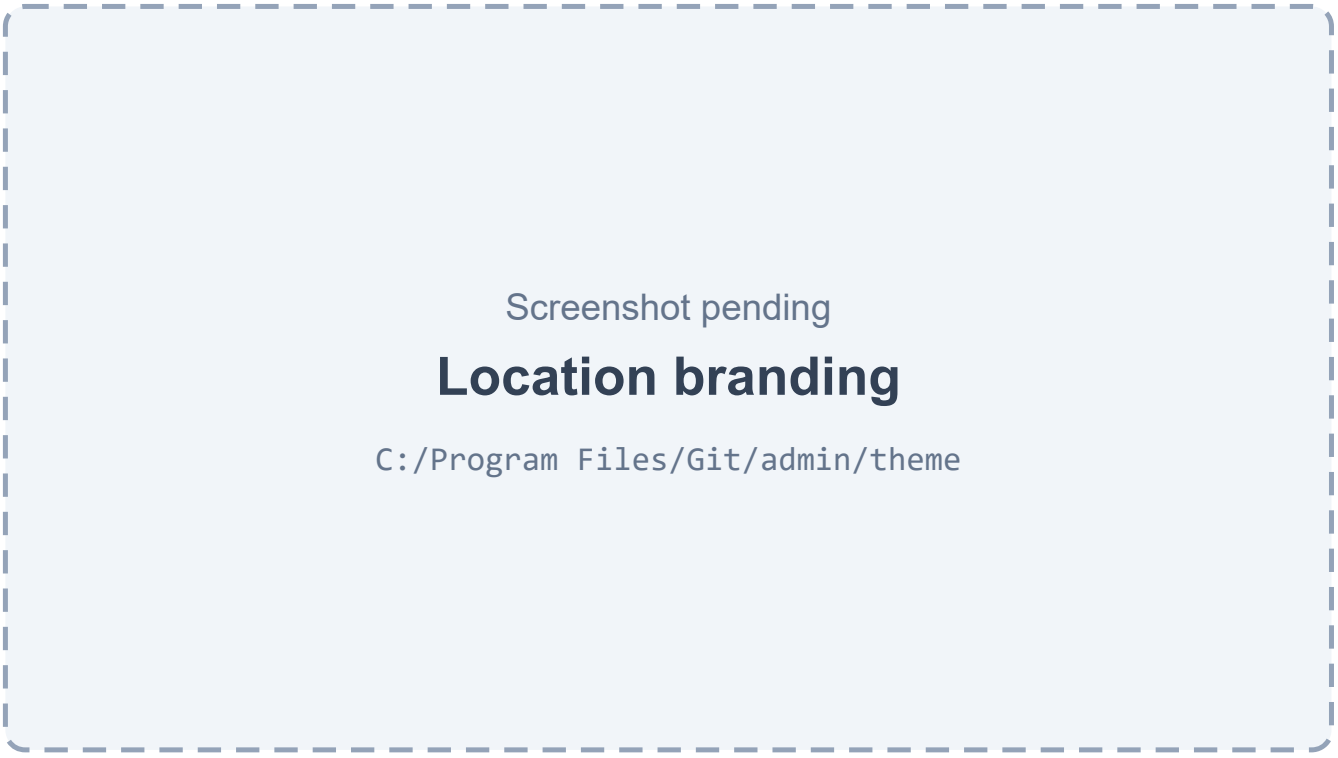


Figure 58.6 — Location branding (screenshot pending).

The sheet is titled "Location branding" with the text "A branch can show its own name and logo. Empty fields use the clinic's." It opens from the button on the theme page. A "Location" list chooses the branch, then an "Overrides" section holds "Name at this location", "Logo (light)" and "Logo (dark)". Saving applies the override; empty fields inherit.

Endpoints.

Action	Endpoint	Permission
Resolved theme for the signed-in clinic	`GET /theme`	any member
Organization view and save	`GET` and `PUT /admin/theme`	`admin.theme.manage`
Location view and save	`GET` and `PUT /admin/theme/location/:locationId`	`admin.theme.manage`
Platform view and save	`GET` and `PUT /platform/theme`	super admin

Action	Endpoint	Permission
Public theme	`GET /public/theme/:organizationId`	none

The table lists the endpoints. A write sends values by short key (a null or empty text clears the value, which inherits) and optional locks (organization and platform only; a location cannot lock). Values are validated by the registry's schema for each key. A write to a key locked higher up fails with 403 `SETTING_LOCKED`. The write takes a row lock, increases the row's version, writes an audit row and invalidates the settings cache in every process through Redis; without Redis the other processes catch up within the 60-second cache time.

Data read and written. `config_values` (one row per scope, scope id and key, with the value, a lock flag, a version and the editor), `audit_logs`.

Technical references: `backend/src/settings-registry/theme/theme.controller.ts` · `ThemeService` · `theme.settings.ts` · `backend/src/settings-registry/settings-registry.service.ts` · `SettingsRegistryService` · `web-app/src/features/theme/` · `web-app/src/features/theme/lib/color.ts`

58.8 Settings registry

Status: [Partial]

The settings registry (`config_values`) is the general store for new settings with the platform, organization and location cascade and locks. Today only the seven theme keys are defined in it. Older settings live in their own typed tables, for example the queue policies and the location billing settings, and keep working unchanged. The code does not move them into the registry.

58.9 Limitations and known gaps

- The sidebar layout is per clinic only; per-role layouts are not written yet (the column exists).
- The owner cannot add new sidebar pages or permissions; both are fixed in code.
- Only the name and logos can be customized per location.
- The brand colour check is advice, not a rule.
- Fonts, spacing and corner radius cannot be customized.

58.10 Owner's FAQ

Who can switch a feature on that my plan does not include? Only the platform team, by entitling it. The owner can only enable features the clinic is entitled to.

If I hide a page in the sidebar, can people still open it? Yes, by address, if their role allows it. Hiding is cosmetic; use roles to remove access.

Can each branch have its own logo? Yes. Each branch can have its own name and light and dark logos; the colour and mode are clinic wide.

Why can I not change the colour? The platform team may have locked that key for clinics. The screen marks such a key as locked at a higher level.

Where do I set what a role sees first? On the role's Home page tab. The clinic-wide fallback is on the Sidebar screen.

What happens to people when I turn a feature off? They lose the permissions that belong to the feature at once, and the dialog before you confirm lists the roles affected.

Chapter 59 — Notification mechanics

This chapter explains how a notification travels inside the Platform, from the business event that causes it to the bell icon where a staff member reads it. It is written for the product owner who wants to understand the pipeline and its guarantees. The Notifications module chapter (Chapter 44) describes the settings screen, the event catalog and what users see; this chapter covers only the mechanics. Today the Platform delivers notifications **in the application only**: WhatsApp, SMS and email are prepared in the code but have no real sender.

59.1 The pipeline

Status: [Implemented] for in-app delivery. [Not implemented] for WhatsApp, SMS and email.

A **notification event** is a named thing that happens, such as `order.submitted` or `item.ready` (21 are seeded, all about lab orders, shipments, stages, appointments linked to labs and payments). When a workflow step runs the `notify` effect, the Platform decides who should be told, whether each person wants it, what the message says and when it may be sent. The result is one row per person and channel in a table called `notification_outbox`. A separate loop in the worker process sends those rows later. This is the **transactional outbox** pattern: the notification row is written in the same database transaction as the business change, so it exists only if the change was committed.

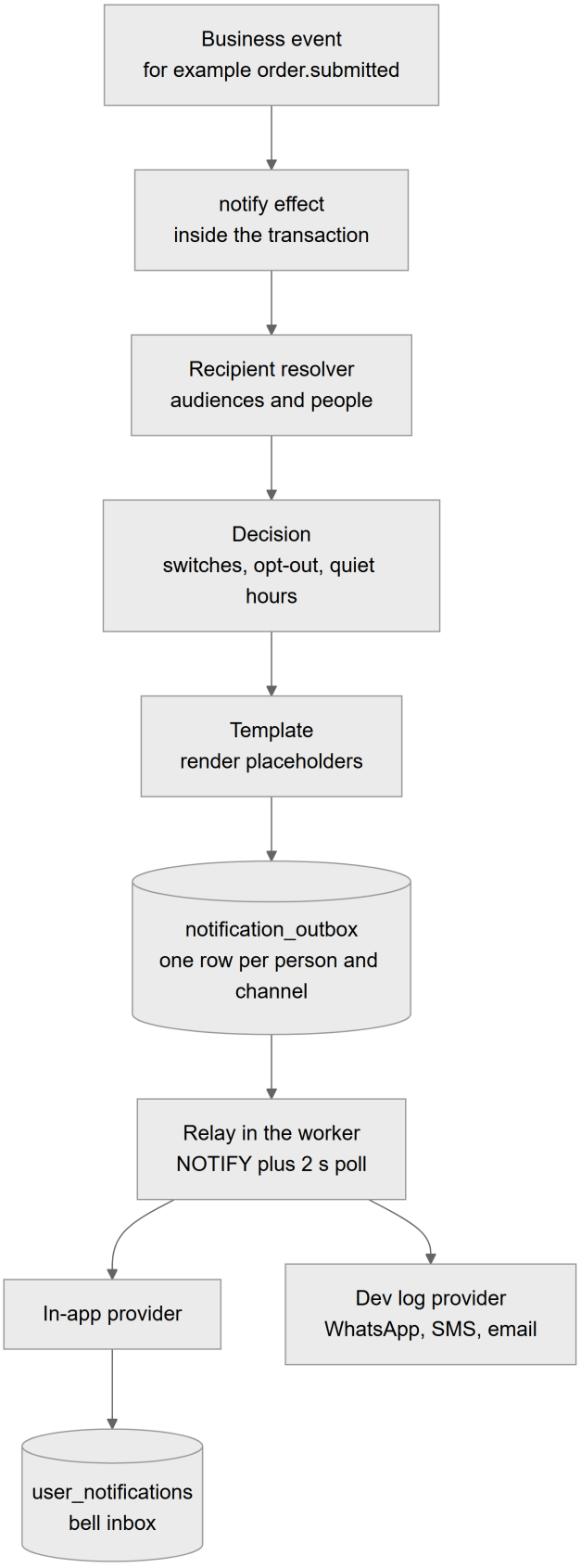


Figure 59.1 — The path of a notification from the business event to the inbox.

The flowchart reads from top to bottom. A "Business event" (for example `order.submitted`) triggers the "notify effect", which runs inside the transaction of the change. The effect asks the "Recipient resolver" for the audiences and people to tell. For each person and channel a "Decision" step applies the switches, the patient's opt-out and quiet hours. A "Template" step renders the message, filling placeholders. The result is stored in the database table `notification_outbox`, one row per person and channel. The "Relay in the worker", woken by a database

NOTIFY and by a two-second poll, claims due rows. The "In-app provider" writes the user's copy into `user_notifications`, which is the bell inbox. A second branch, the "Dev log provider", handles WhatsApp, SMS and email by writing the message to the server log instead of sending it.

Technical references: `backend/src/notifications/services/notification.service.ts` · `NotificationService.enqueue` · `backend/src/notifications/effects/notify.effect.ts` · `backend/src/notifications/delivery/`

59.2 Event, recipients and audiences

Status: [Implemented]

The `notify` effect names an event code (and may narrow the audiences). An unknown code fails the whole transition, so a typo can never silently drop messages. Who receives the message is decided by a **recipient resolver** registered per kind of subject: the module that owns the subject says how to find its doctor, coordinator, lab and patient. Today only the lab order workflow registers one, so lab events are the only ones that produce notifications.

There are four **audiences**: patient (the only external audience), doctor, coordinator and lab. A recipient is a user (a signed-in person), a patient or a lab contact. The same person listed twice, or under two audiences, gets one row per channel. If an event is not meant for an audience, enqueueing for it fails with `NOTIFICATION_AUDIENCE_INVALID`. A patient is looked up in the same organization, so a clinic can never message another clinic's patient.

Technical references: `backend/src/notifications/effects/notification-recipients.registry.ts` · `NotificationRecipientsRegistry` · `backend/src/notifications/notifications.enums.ts`

59.3 Rules and the decision order

Status: [Implemented]

For every recipient and channel the Platform makes one decision. The first check that matches wins:

1. An internal audience with a mandatory event on the in-app channel is queued, ignoring settings.
2. If the audience's master switch is off, the row is stored as `skipped_disabled`.
3. If the rule for this event and channel is off, it is `skipped_disabled`.
4. If the patient opted out of the channel, it is `skipped_opt_out`.
5. If the time is inside the audience's quiet hours (and the channel is not in-app), it is `held_quiet_hours` until the window ends.
6. Otherwise it is `queued` for now.

Skipped rows are still written, so the clinic can see in its message log why nothing was sent. Defaults apply when a clinic has set nothing: the patient audience and every patient rule are off; internal audiences are on, with in-app on and the external channels off. The channels offered per audience in code are in-app only for doctor, coordinator and lab, and none for patients, so patients receive nothing today.

Quiet hours are a start and end time per audience. The window can cross midnight (for example 22:00 to 08:00); equal start and end means no window. They use the audience setting's timezone if it names one, otherwise the timezone of the location the event belongs to (Chapter 62), otherwise UTC. In-app messages are never held, because they are silent.

Technical references: `backend/src/notifications/decision/notification-decision.ts` · `decide` · `quietHoursEnd`

59.4 Templates

Status: [Implemented]

A template has a subject, a body, a channel, a language (English, Urdu or Roman Urdu) and optional provider reference. The body uses placeholders such as `{clinic_name}` and `{recipient_name}`, plus the event's own. An unknown placeholder becomes empty text so no raw braces ever reach a person. The Platform picks the clinic's own template in the person's language, then the platform's template in that language, then the same two in English. A rule can name a specific template. If no template exists, the event's name and description are used.

Technical references: `backend/src/notifications/templates/template-render.ts` · `renderTemplate` · `pickTemplate`

59.5 Outbox, relay and sending

Status: [Implemented]

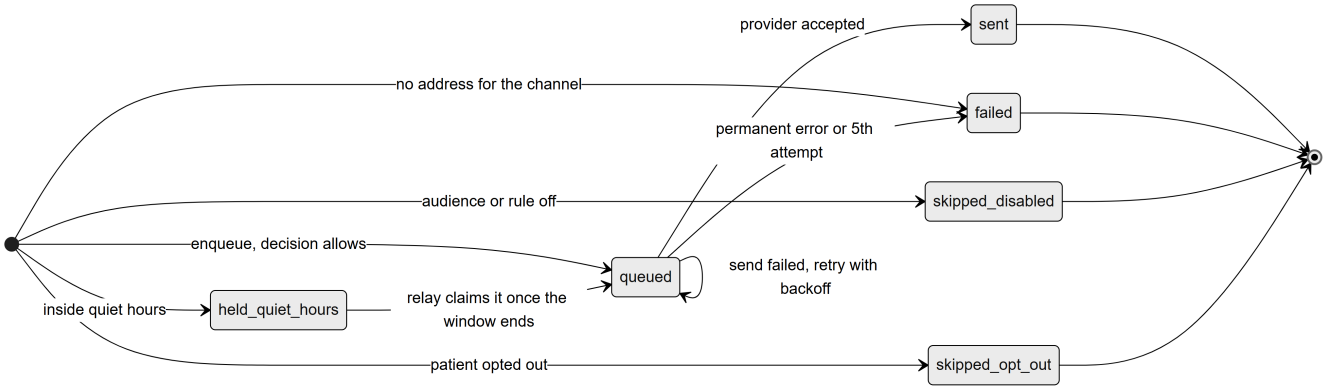


Figure 59.2 — Status lifecycle of a row in `notification_outbox`.

The diagram starts at a dot and shows several entry points. A row is created as `queued` when the decision allows sending, as `held_quiet_hours` when it falls inside quiet hours, as `skipped_disabled` when a switch or rule is off, as `skipped_opt_out` when the patient opted out, and directly as `failed` when the channel needs an address and the recipient has none. A held row becomes `queued` when the relay claims it once the quiet window has ended. A queued row becomes `sent` when the provider accepts it. A queued row whose send fails stays `queued` and is tried again after a pause. It becomes `failed` on a permanent error or at the fifth attempt. `sent`, `failed` and both skipped statuses are final. The status `delivered` exists in the code for providers that report delivery but nothing sets it today.

The relay (`NotificationRelayService`) runs only in processes that run jobs, which is the worker. It is woken at once by a database NOTIFY sent inside the enqueue transaction (so it fires only on commit) and also polls every two seconds, which picks up retries and quiet-hours rows that are due. If the NOTIFY connection fails, it logs a warning and relies on polling.

Sending works like this:

- A row is **claimed** with one short statement that locks it so no other worker takes it, adds one to its attempt count and pushes its due time out by a five-minute lease. If a worker dies mid-send, the row reappears after the lease.
- The provider is called with no transaction open; the result is written back only if this claim is still the latest.

- A failure is retried after 30 seconds, 2 minutes, 10 minutes and one hour; the fifth failure makes the row `failed`. A provider can mark an error permanent to skip retries.
- Delivery is at least once. The in-app copy is protected by a unique link to the outbox row, so a retry after a crash never shows the same message twice.
- Batches of 50 are processed until nothing is due.

Providers. The in-app provider writes `user_notifications` (user, event, title, body, link, read time). The development provider logs WhatsApp, SMS and email messages and marks them sent, so flows run end to end; it sends nothing to anyone. A real provider can take over a channel through the provider registry.

Technical references: `backend/src/notifications/delivery/notification-relay.service.ts` · `notification-dispatcher.service.ts` · `in-app.provider.ts` · `dev-log.provider.ts` · `channel-provider.registry.ts`

59.6 The inbox and the settings screen

Status: [Implemented]

The bell shows the signed-in user's inbox through `GET /me/notifications` (paged, with an unread-only filter), `GET /me/notifications/unread-count`, `POST /me/notifications/:id/read` and `POST /me/notifications/read-all`. These routes are context free: a user's inbox spans every clinic they work at, and lab users have no clinic location. Clinic settings are under `/notification-settings` and need `notifications.settings.manage`: the overview with master switches and quiet hours, audience updates, the rules matrix, templates (create, change, reset to platform wording) and the message log. A patient's channel preferences are read with `patients.view` and changed with `patients.edit`. Chapter 44 describes these screens.

59.7 Limitations and known gaps

- Only lab-related events exist; no appointment reminders or billing notices are produced.
- WhatsApp, SMS and email have no provider; patients cannot be reached.
- Only the lab workflow registers recipients.
- Inventory alerts are not notifications; they go to the server log only (Chapter 63).
- The status `delivered` is never set.

59.8 Owner's FAQ

If a lab order is rolled back, is a notification still sent? No. The notification row is part of the same transaction and disappears with it.

Why did a staff member not get a message? Check the log in notification settings: it shows each row with its status and reason, such as an audience switched off or a rule disabled.

Can I send reminders to patients by WhatsApp? Not yet. The decision logic and templates exist, but the channel is commented out and no provider is connected.

What happens if the worker is down? Rows stay queued in the database and are sent when the worker returns; nothing is lost.

Can quiet hours delay an in-app message? No. Quiet hours apply only to external channels.

Chapter 60 — Audit logging

This chapter explains how the Platform records who changed what and when: the history tables, the modules that write to them today, who may read them and what is kept for how long. It is written for the product owner, who must be able to answer an auditor's question honestly. The most important message is that audit coverage is **uneven**: patients and several administration areas are recorded in detail, while appointments, employees, payroll and organizations are not yet recorded in the general audit log.

60.1 The history tables

Status: [Implemented]

The Platform has several history tables, written by different modules at different times.

Table	Written by	What a row says
`audit_logs`	the shared audit service	One change: entity type and id, action (create, update, void, delete, export, view_sensitive), the **changed fields** before and after, actor, actor side (clinic, lab, platform, system), IP address and request id
`status_history`	the shared audit service	One state change: from state, to state, action code, actor, channel (app, portal, magic link, proxy, QR), reason
`billing_audit_logs`	billing services	One billing event with before and after, reason, amount, actor and the actor's role
`queue_events`	the queue orchestrator	One queue event with a running sequence number, status before and after
`prescription_history`	the prescription service	One prescription event: created, amended (with a full snapshot of the earlier version), finalized, printed, shared

The table lists the five history tables and what each row records. `audit_logs` and `status_history` are the general tables. The other three are older, module-specific histories that stay as they are.

Append only. `audit_logs` and `status_history` are protected by database triggers that reject any update or delete, so a row, once written, cannot be changed through the application or by accident. `commission_entries` has a similar trigger (Chapter 35). `billing_audit_logs`, `queue_events` and `prescription_history` are append-only by convention: the code only inserts, but no trigger enforces it.

Written in the same transaction. The shared service always writes in the caller's transaction, so the history commits or rolls back together with the change. An update that changed nothing writes no row.

Subject. Each general row can carry a `subject_id`, the record whose history it joins. For example, a patient's vitals row carries the patient as subject, so one query returns the patient's whole history.

Technical references: `backend/src/common/audit/audit.service.ts` · `AuditService` · `backend/src/common/audit/entities/` · `backend/src/migrations/1792490000000-lab-management-foundation.ts` · `backend/src/billing/audit/billing-audit.service.ts` · `backend/src/queue/entities/queue-event.entity.ts` · `backend/src/prescriptions/entities/prescription-history.entity.ts`

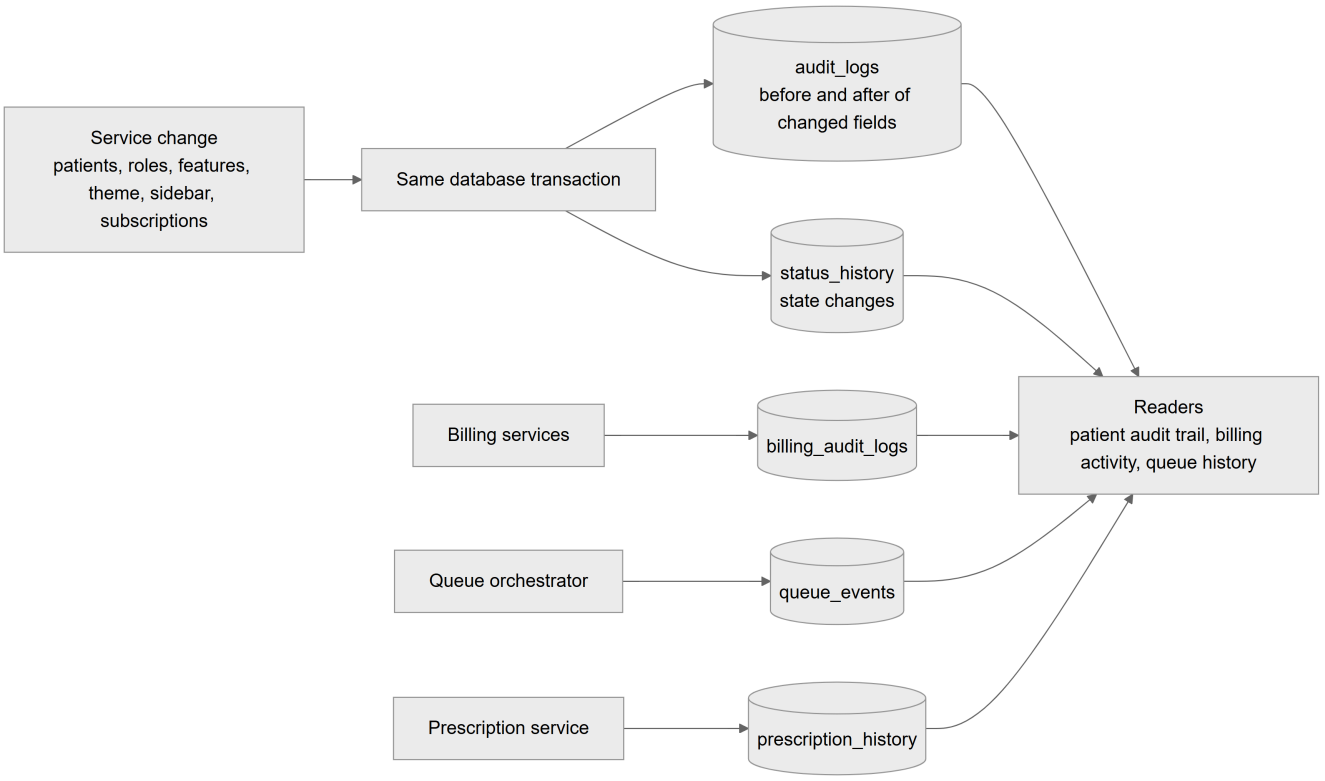


Figure 60.1 — Writers, history tables and readers.

The flowchart reads left to right. A box "Service change" (patients, roles, features, theme, sidebar, subscriptions) leads to "Same database transaction", which writes two tables: `audit_logs` (before and after of changed fields) and `status_history` (state changes). Three other writers stand alone: "Billing services" write `billing_audit_logs`, the "Queue orchestrator" writes `queue_events` and the "Prescription service" writes `prescription_history`. All five tables feed a single box, "Readers", which names the patient audit trail, the billing activity feed and the queue history.

60.2 What is recorded today

Status: [Partial] — coverage differs by module. The table states what the code does now.

Area	Written to the general audit log?	Detail
Patients	Yes, [Implemented]	Create, update and delete of the patient; activate and deactivate (as status history); 13 kinds of patient records (dental info, tooth chart, vitals, treatment plans and plan items, prescriptions as listed on the patient, documents, consents, communications, recalls, referrals, feedback and insurance). Delete rows keep the last snapshot and the counts of linked records. Patient import and queue walk-in patient creation are audited the same way. File addresses and signatures are never stored in the log.
Patient exports	Yes	Every patient CSV export writes an `export` row with the number of rows and the filters used
Report exports	Yes	Every Excel or PDF report export writes an `export` row with the report, format and period
Employment roles	Yes	Each create, edit, clone and delete, with before and after values and the reason for high-risk grants

Area	Written to the general audit log?	Detail
Organization features and location overrides	Yes	Each switch, with the platform recorded as actor when a super admin acts
Subscriptions and subscription payments	Yes	Payments and status changes
Theme and registry settings	Yes	One row per changed key
Sidebar layout	Yes	Save and reset
Billing	Own table	`billing_audit_logs` for charges, invoices, payments, refunds, discounts, cash close and settings
Lab modules	Yes	Lab orders, items, attachments, shipments, production, links, members, catalog, patient lab invoices, credits, vendor bills, statements and payments
Workflow engine	Yes	Transitions write status history; form schemas, workflow and stage templates write audit rows
Queue	Own table	`queue_events`, written for every queue change
Prescriptions	Own table	`prescription_history`
Appointments	**No** [Not implemented]	No audit row is written for booking, rescheduling, cancelling or status changes
Employees (staff records, salary, schedules)	**No** [Not implemented]	Only role definitions are audited, not employee data
Payroll	**No** [Not implemented]	No audit rows for payroll records or payments
Organizations and locations	**No** [Not implemented]	Settings and details of organizations and locations are not audited, apart from features, theme and subscriptions
Encounters, treatments, inventory	**No** [Not implemented]	Not audited
Sign-in events	Separate	`security_logs` (Chapter 56)

The table lists each area, whether it writes to the general audit log and what is recorded. It was built by searching the code for every writer of the audit service. The audit rollout plan names appointments, staff, payroll and organizations as later phases; their code is not written yet.

Technical references: `backend/src/patients/patient-audit.ts` · `recordPatientChange` · `backend/src/patients/patients.service.ts` · `backend/src/reports/reports.controller.ts` · `backend/src/employees/employment-roles.service.ts` · `backend/src/feature-flags/feature-admin.service.ts` ·

`backend/src/settings-registry/settings-registry.service.ts` · `backend/src/subscriptions/subscriptions.service.ts`

60.3 Reading the history

Status: [Partial]

Reader	Endpoint	Permission	Screen
Patient audit trail	`GET /patients/:id/audit-trail`	`admin.audit.view`	A history sheet on the patient page, shown only to holders of the permission
Queue history of a session	`GET /queue/sessions/:id/events`	`admin.audit.view`	Queue history
Billing activity feed	`GET /billing/activity`	`billing.activity.view`	Billing activity page
Entity timelines for labs	lab endpoints	lab permissions	Activity timelines on lab pages

The table lists the readers. The patient audit trail merges `audit_logs` and `status_history` for the patient and every record under them, newest first (up to 200 entries), and the values are redacted for readers who lack the clinical or document permissions. The permission `admin.audit.view` is "View the audit log" in the Admin group; the owner and super admin hold it, and the Admin system role holds the whole Admin group by default (Chapter 11).

There is **no organization-wide audit page**: no screen or endpoint lists all changes of a clinic, searches by user or date, or exports the log. The general tables can be read only through the entity or subject timelines in the table and, for developers, in the database.

60.4 Retention

Status: [Not implemented]

The Platform has no retention or purge policy for audit data. The audit tables are append-only, so nothing deletes them. The only clean-up job in the area removes queue idempotency keys older than 24 hours and published realtime outbox rows older than 7 days (Chapter 63), which are technical records, not history. `security_logs` and session rows are also never purged. A clinic with a legal retention limit has no built-in way to apply it.

60.5 Who is the actor

Each general row stores the user id, the **actor side** and, when available, the IP address and a request id taken from an `x-request-id` header if the caller sent one. The side is clinic for clinic staff, lab for lab members, platform for a super admin and system for background jobs. A super admin acting inside a clinic is recorded as platform.

60.6 Limitations and known gaps

- Appointments, staff data, payroll and organization settings have no audit history.

- No organization-wide audit viewer, filter or export.
- No retention policy.
- Three of the five history tables are append-only by convention only.
- Reading sensitive patient data is not logged; the action `view_sensitive` exists but is not written by any reader.
- Encounter notes in progress are not audited; the plan is one snapshot at completion (not built).

60.7 Owner's FAQ

Can I see who edited a patient's phone number? Yes, if you hold `admin.audit.view`: open the patient and the audit history. It shows the old and new value, who and when.

Can I see who cancelled an appointment? Not from an audit log today. Appointments are not audited. The appointment row itself shows its status and cancellation reason.

Can someone delete a history row to hide a change? Not through the application. For `audit_logs` and `status_history` the database itself rejects updates and deletes.

Who is told when patient data is exported? Nobody is notified, but each export writes a record with who, when, how many rows and the filters.

How long are audit records kept? Forever, because nothing deletes them.

Chapter 61 — Error handling

This chapter describes how the Platform reports problems: the shape of every API answer, the families of error codes, how validation and database conflicts become messages, how retries stay safe, and what the web app shows the user. It is written for the product owner and for support staff who read an error message and need to know what it means. Chapter 5 traces one request through the system and shows the main status codes; this chapter goes into the error side in more detail.

61.1 The envelope

Status: [Implemented]

Every controller answers inside one JSON envelope built by `ResponseWrapper`. On success the fields are `data` (the payload), `message` (for example "Patient created successfully"), `isSuccess` (true) and `httpStatusCode` (always 200 in the envelope, even when the HTTP status is 201). On failure the global exception filter `GlobalExceptionFilter` builds the same envelope with `isSuccess` false, the real HTTP status in `httpStatusCode`, a short `message` for people, a `devMessage` with a technical text for developers and an `error` field holding the exception body. The web app reads `res.data.data` on success and `err.response.data.message` (or `error.code`) on failure.

Technical references: `backend/src/common/dtos/ResponseWrapper.ts` · `backend/src/common/exceptions/HttpExceptionHandler.ts`

61.2 What the filter does with an error

Status: [Implemented]

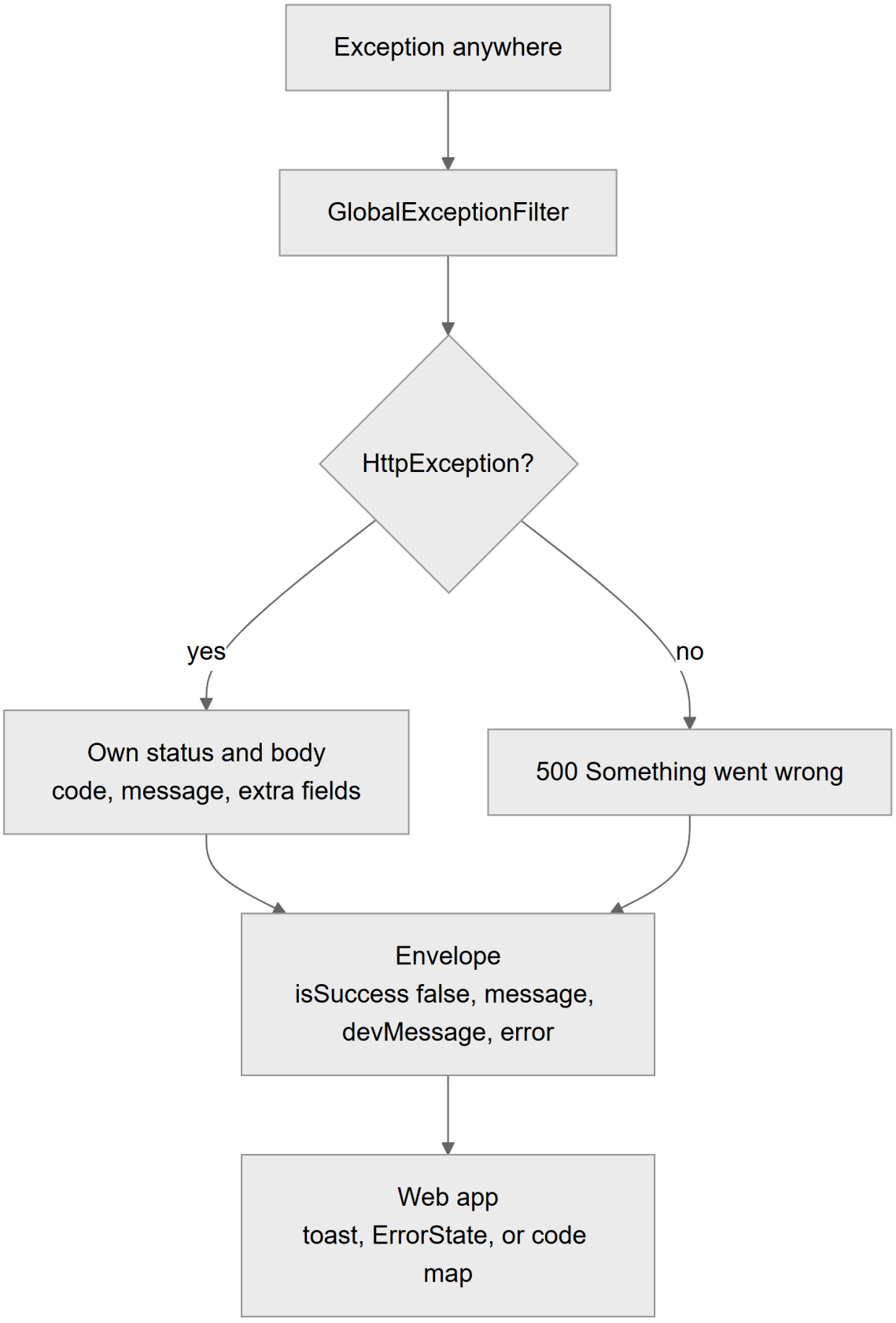


Figure 61.1 — How an exception becomes an envelope and a message on screen.

The flowchart reads from top to bottom. An "Exception anywhere" in the request reaches `GlobalExceptionFilter`. The filter asks whether it is an HTTP exception. If yes, the request keeps its own status and body, with a code, a message and any extra fields. If no, which means an unexpected failure, the answer is

500 with the message "Something went wrong". Both branches build the envelope: `isSuccess` false, `message`, `devMessage` and `error`. The web app then shows a toast, an `ErrorState` panel or a friendly text chosen by the error code.

Categories of error:

- **Expected HTTP errors.** Business rules and security refusals are thrown as typed exceptions with a status and, in most modules, a machine-readable `code`. The message is written for the user.
- **Validation errors.** The global validation pipe rejects bad input (section 61.3). Several messages are possible; the envelope's `message` shows the first one.
- **Unexpected errors.** Anything that is not an HTTP exception, such as an unhandled database failure, is logged with its stack on the server and answered as 500 "Something went wrong". The user-facing text never contains internals.
- **Rate limit.** A throttled request is answered 429 with "Too many requests. Please try again later." and a number of seconds to wait (Chapter 56).

Every error is logged by the application logger together with the request address.

61.3 Validation

Status: [Implemented]

The global validation pipe converts input to the types named in each request class and has `whitelist` and `forbidNonWhitelisted` on. A request that contains a field the API does not define is **rejected** with 400, not silently ignored, and a field with a wrong type or format is rejected with a message such as "email must be an email". Request classes use validation decorators for lengths, formats, enumerations and nested objects. The same rules appear in the web app as form schemas, so most mistakes are caught before the request leaves the browser; the server rule is the one that counts.

61.4 Error code families

Status: [Implemented]

Status	Code or family	Meaning
400	`IDEMPOTENCY_KEY_REQUIRED`, `REASON_REQUIRED`, `INVALID_PERMISSIONS`, validation messages	The request is incomplete or malformed
401	(message only)	No token, expired token or bad token; the web app tries to refresh
402	`SUBSCRIPTION_EXPIRED`	The clinic is read-only because its subscription lapsed
403	`NO_PERMISSION` with the missing permissions, `NOT_ENTITLED`, `ESCALATION`, `SYSTEM_ROLE_READ_ONLY`, `SETTING_LOCKED`, `FORBIDDEN`, `LAB_ACCESS_DENIED`	The caller is not allowed
404	`NOT_FOUND`, `BILLING_NOT_FOUND`, `LAB_NOT_FOUND`, and similar	The record does not exist in this organization
409	`DOCTOR_DOUBLE_BOOKED`, `VERSION_CONFLICT`, `STALE_VERSION`, `ALREADY_CHECKED_IN`, `ACTIVE_WITH_OTHER_DOCTOR`, `DOCTOR_BUSY`, `ROLE_NAME_TAKEN`, `ROLE_HAS_MEMBERS`, `REQUEST_IN_FLIGHT`, `BILLING_DUPLICATE`, `PLAN_LIMIT` and similar	The request conflicts with the current state or with another request

Status	Code or family	Meaning
422	`INVALID_TRANSITION`, `IDEMPOTENCY_KEY_REUSED`, `NOT_TODAY`, `LATE_LIMIT_EXCEEDED`, `SESSION_NOT_OPEN`, many `BILLING_*`, `FORM_*` and `WORKFLOW_*` codes	The request is well formed but the rules refuse it
429	`MAGIC_LINK_RATE_LIMITED` and the login throttle	Too many requests
500	`BILLING_INVARIANT_VIOLATION`, `WORKFLOW_MISCONFIGURED`, anything unexpected	A bug or a broken money rule
503	`LOCK_TIMEOUT`	The queue was busy; retry

The table groups the codes by HTTP status and shows what each family means. It lists examples, not every code. The codes are thrown by several module-specific error builders, each of which passes the code in `error.code`:

- **Billing** (`billing-errors.ts`): about 40 `BILLING_*` codes plus `COMPENSATION_*` and `COMMISSION_*` for the commission rules. Each maps to a fixed status, for example allocation larger than the balance is 422 and a duplicate is 409.
- **Queue** (`queue.errors.ts`): `NOT_FOUND`, `FORBIDDEN`, `INVALID_TRANSITION`, `SESSION_NOT_OPEN`, `DOCTOR_BUSY`, `VERSION_CONFLICT`, `ALREADY_CHECKED_IN`, `ACTIVE_WITH_OTHER_DOCTOR`, `LATE_LIMIT_EXCEEDED`, `NOT_TODAY`, `LOCK_TIMEOUT`, `IDEMPOTENCY_KEY_*`, `REQUEST_IN_FLIGHT`, `REASON_REQUIRED`.
- **Workflow engine** (`engine-errors.ts`): form, workflow and stage template codes, `INVALID_TRANSITION`, `TRANSITION_FORBIDDEN`, `TRANSITION_INPUT_REQUIRED`, `TRANSITION_REASON_REQUIRED`.
- **Labs** (`lab-errors.ts`, plus lab billing, catalog, orders and payables): `LAB_*` codes, including `LAB_PORTAL_CLINIC_CONTEXT` and `LAB_SELECTION_REQUIRED`.
- **Notifications** (`notification-errors.ts`): `NOTIFICATION_*` codes and `PATIENT_NOT_FOUND`.
- **Access and features**: `NO_PERMISSION`, `ESCALATION`, `INVALID_PERMISSIONS`, `NOT_ENTITLED`, `SETTING_LOCKED`, `PLAN_LIMIT`.

`DOCTOR_DOUBLE_BOOKED` is the appointment overlap error (Chapter 29): the database rejects two bookings of one doctor at overlapping times, and the service turns the failure into a 409 with that code.

Technical references: `backend/src/billing/common/billing-errors.ts` · `backend/src/queue/queue.errors.ts` · `backend/src/engine/common/engine-errors.ts` · `backend/src/labs/common/lab-errors.ts` · `backend/src/notifications/notification-errors.ts` · `backend/src/access/decorators/access.decorators.ts`

61.5 Safe retries: idempotency keys and If-Match

Status: [Implemented]

Two mechanisms protect against double clicks, flaky networks and two people editing the same record.

Idempotency-Key. A client sends a header `Idempotency-Key` (1 to 150 characters) on actions that must not run twice. The server stores the key, a hash of the request and, once finished, the response, inside the same transaction as the work. If the same key arrives again with the same request, the stored answer is replayed and nothing runs twice. The same key with a different request gives 422 `IDEMPOTENCY_KEY_REUSED`. The same key while the first request is still running gives 409 `REQUEST_IN_FLIGHT`. A missing key where one is required gives 400 `IDEMPOTENCY_KEY_REQUIRED`. The key is required on every queue change, on patient check-in from an appointment, on money-moving lab billing posts and on the workflow engine transitions; the web app creates a fresh key for each open sheet or action. Queue idempotency keys are deleted after 24 hours by a nightly job.

If-Match and versions. Records that two people may edit carry a version number. A client may send `If-Match: <version>` (queue entries, lab order actions) or the version in the body; the server checks it while holding a row lock and, if the record has moved on, answers 409 with `VERSION_CONFLICT` or `STALE_VERSION` and the current

state or version. The web app reloads and asks the user to try again, with texts such as "Someone else changed this just now. The latest version is loaded".

Technical references: `backend/src/common/idempotency/idempotency.service.ts` · `IdempotencyService` · `backend/src/common/errors/optimistic-lock.ts` · `assertVersion` · `backend/src/queue/decorators/queue-request.decorators.ts`

61.6 Database constraints become messages

Status: [Partial]

The database enforces uniqueness and exclusion rules, and several services translate the database error into a clear conflict. The translation is per service; there is no global mapping.

- **Exclusion constraint on appointments** (`no_doctor_overlap`): Postgres error 23P01 becomes 409 `DOCTOR_DOUBLE_BOOKED` . Compensation plan versions and lab prices use similar exclusion constraints with their own messages.
- **Unique violations** (23505) are translated in billing, lab billing, lab orders, lab catalog and lab payables into codes such as `BILLING_DUPLICATE` .
- **Foreign key violations** (23503) are translated when deleting an employee that is still referenced.
- **Lock timeouts** in the queue become 503 `LOCK_TIMEOUT` .

A database error that no service translates reaches the filter as an unexpected error and is answered as 500 "Something went wrong". A duplicate patient number or purchase order number, for instance, would currently surface this way (Chapter 62).

61.7 In the web app

Status: [Implemented]

- **401 and refresh.** The shared axios client refreshes the access token on a 401 and retries once (Chapter 56).
- **Toasts.** Mutations report failures with a toast (the library `sonner` , at the top right, closable, 4 seconds). Many hooks have a helper that reads `err.response.data.message` and falls back to a text such as "Failed to create item". About 180 places raise an error toast.
- **Friendly code maps.** Billing, labs, lab orders, lab catalog, lab payables, lab billing and the engine studio each have a small map from an error code to a plain sentence, for example the billing code for a missing document code becomes "Set a document code for this location in Billing settings first." If a code has no entry, the server's own message is shown, then a generic text.
- **Stale screens.** The queue hook shows "Someone else just changed this patient — showing the latest." as a warning on `VERSION_CONFLICT` and refreshes the screen.
- **Page states.** Every list or detail screen uses `LoadingSkeleton` while loading (table, cards, stats or detail shape), `EmptyState` when there is nothing (icon, title, message and an optional action) and `ErrorState` when a request failed (title "Something went wrong", a sentence and a **Try again** button). A route-level error boundary, `RouteError` , shows the error message with a retry button for render failures.
- **Access errors.** A missing route permission shows the No access page, not an error (Chapter 57). The access gate shows "Couldn't load your access" if `/me/access` fails.
- **Query settings.** Queries retry twice and mutations once, with 5 minutes of freshness.

Technical references: `web-app/src/lib/api.ts` · `web-app/src/lib/query-client.ts` · `web-app/src/features/billing/lib/billing-errors.ts` · `web-app/src/features/engine-studio/lib/studio-errors.ts` · `web-app/src/features/queue/hooks/useQueueAction.ts` · `web-app/src/components/feedback/ErrorState.tsx` · `EmptyState.tsx` · `LoadingSkeleton.tsx` · `web-app/src/components/shared/RouteError.tsx`

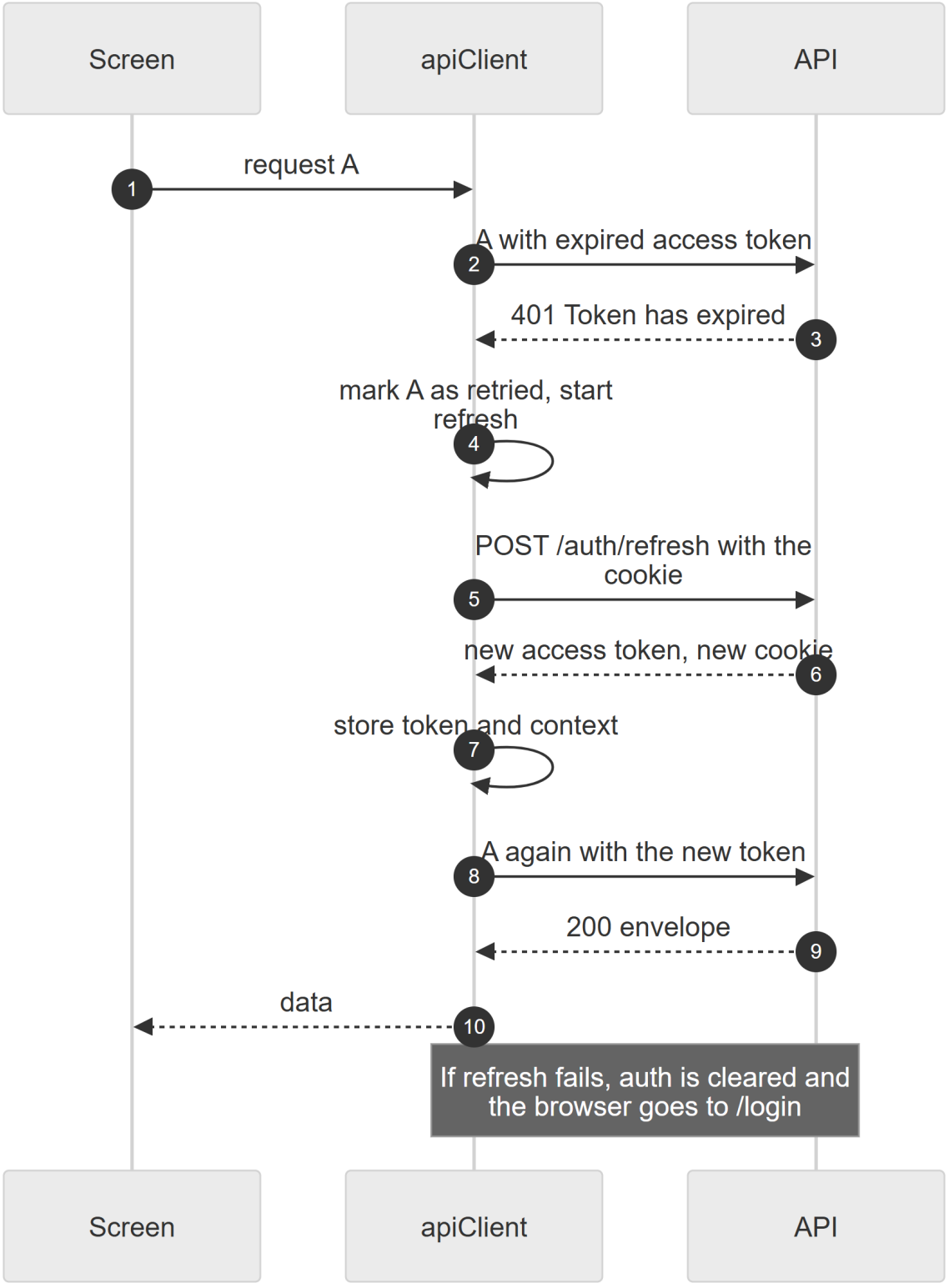


Figure 61.2 — How an expired access token is replaced without the user noticing.

The diagram has three participants: the Screen, the `apiClient` axios instance and the API. The screen sends request A. The client sends A with an expired access token. The API answers 401 "Token has expired". The client marks A as retried and starts a refresh. It sends `POST /auth/refresh` with the cookie. The API answers with a new

access token and a new cookie. The client stores the token and context, sends A again with the new token, gets a 200 envelope and returns the data to the screen. A note says that if the refresh fails, the stored login is cleared and the browser goes to `/login`. While one refresh is running, other requests that fail with 401 wait in a queue and are retried with the new token.

61.8 Limitations and known gaps

- Error messages are in English. Only the No access headline carries a Roman-Urdu line.
- Code-to-sentence maps exist for some modules only; others show the server's text.
- There is no central error-tracking service; errors are written to the server log.
- Database errors are translated service by service, so an untranslated one shows as a generic 500.
- The error body includes a developer message field in every failure.

61.9 Owner's FAQ

A user says "Something went wrong". What now? It is an unexpected server error. Ask for the time and the page; the server log holds the cause. Other failures give a specific message.

What does 402 mean? The subscription expired; the clinic is read-only until a payment is recorded.

What does "Someone else changed this" mean? Another person saved the same record first. The latest data is loaded; redo the change.

What happens if the receptionist double-clicks "Check in"? The second click carries the same idempotency key, so the first result is returned and only one token is issued.

Does the API ignore fields it does not know? No. It rejects them with 400.

Chapter 62 — Time, money and document numbers

This chapter explains three rules that run through every module: how the Platform decides what time it is for a clinic, how it stores and calculates money, and how it numbers invoices, patients, orders and other documents. It is written for the product owner, because mistakes in these three areas, a wrong day, a wrong rupee or a duplicate number, are the ones that clinics notice first. The chapter states the rules, where they are followed strictly, and where the code still departs from them.

62.1 Clinic time

Status: [Implemented] for the rule. [Partial] for coverage: some jobs and statistics still use the server clock.

The rule. Every clinic date and time follows the **location's** timezone (the `timezone` of the location, for example a Pakistan or a Gulf timezone). Working hours, appointment availability, the same-day duplicate check, the queue's day boundary, token days and quiet hours all use it. A clinic in one timezone and a server in another still agree on what "today" means for the clinic.

How it works. `ClockService` is the single source of "now" and "today" for clinic logic. Its main functions are:

- `now()` : the current instant (tests can replace it).
- `timezoneFor(locationId)` : the location's timezone name, defaulting to UTC when the stored name is not valid, cached for 60 seconds.
- `today(timezone)` : the location-local date as `YYYY-MM-DD` .
- `wallClock(timezone, instant)` : the local weekday and `HH:MM` .
- `dayRange(timezone, date)` : the start and end instants (in UTC) of a local day.
- `at(timezone, date, time)` : the instant when the clinic's wall clock reads a given date and time.

The code rule is: never use `new Date()` together with `setHours()` for a clinic day or time, because those use the server's timezone. Appointments, queue and notifications follow it (Chapters 29, 30 and 59).

Where the server clock is still used. These places do not use the location's timezone:

- **Scheduled jobs.** Daily and monthly jobs run on the worker's server timezone, for example the 01:00 credit expiry, the 02:30 billing check, the 08:00 inventory alert and the payroll run at 05:00 on the 1st (Chapter 63 and Appendix E). The queue's day-end job is an exception: it decides what a past clinic day is from each location's policy.
- **Credit expiry.** The check works out "today" from the server's local date, so a prepaid credit can expire a few hours early or late for a clinic in another timezone.
- **Statistics.** The financial summary takes its default date and its month list from the server date.
- **Numbering years.** The year inside purchase order and prescription numbers is the server's year.
- **Leave balance year** uses the server year.
- **The lab catalog calendar** fixes its "today" to one specific timezone (Asia/Karachi) regardless of the location.
- **Exports** that print a date (patient export file name) use the server date.

For a clinic whose timezone differs from the server's by a few hours, the differences matter only near midnight or at month boundaries.

Technical references: `backend/src/common/clock/clock.service.ts` · `ClockService` · `backend/src/common/clock/tz.util.ts` · `backend/src/billing/patient-credits.service.ts` · `backend/src/billing/financial-summary.service.ts` · `web-app/src/features/lab-catalog/lib/money.ts` · `catalogToday`

62.2 Money

Status: [Implemented]

What is stored. Money has two storage styles, depending on the module:

Style	Where	Meaning
`numeric(12,2)` rupees	Patient billing: charges, invoices, payments, refunds, ledger entries	Exact decimal with two places
`bigint` paisa	Commissions (rules, entries, settlements, compensation plans), the lab billing tables (invoice, credits, payments), and a few billing fields	Whole number of the smallest unit
`decimal(10,2)`	Inventory prices and quantities, payroll records, employee salary on assignments	Exact decimal

The table shows the three styles. A **paisa** is one hundredth of a rupee; "PKR" is the default currency and not a limit. Amounts cross the API as numbers with at most two decimals in the billing module, and as integer paisa in the lab and commission modules.

How it is calculated. All billing math is done in **integer paisa** so that no floating-point error can creep in. `money.ts` holds the helpers: `toPaisa` and `fromPaisa`, `sumPaisa`, `percentOf` and `basisPointsOf`. A percentage with up to two decimals is turned into basis points (17.5 percent becomes 1750) and applied with exact big-integer arithmetic.

- **Rounding is half up** (0.5 paisa rounds to the next whole paisa), the same everywhere, including tax.
- **Splitting.** When an amount is shared in proportion (for example tax spread over charges), each share is rounded down and the leftover paisa go to the largest share, so the shares always add up exactly to the total.
- **Database columns** are read through transformers: `numeric` comes back from the database as text and is rounded to two decimals; `bigint` comes back as text and is converted to a number, which is exact for any clinic amount.

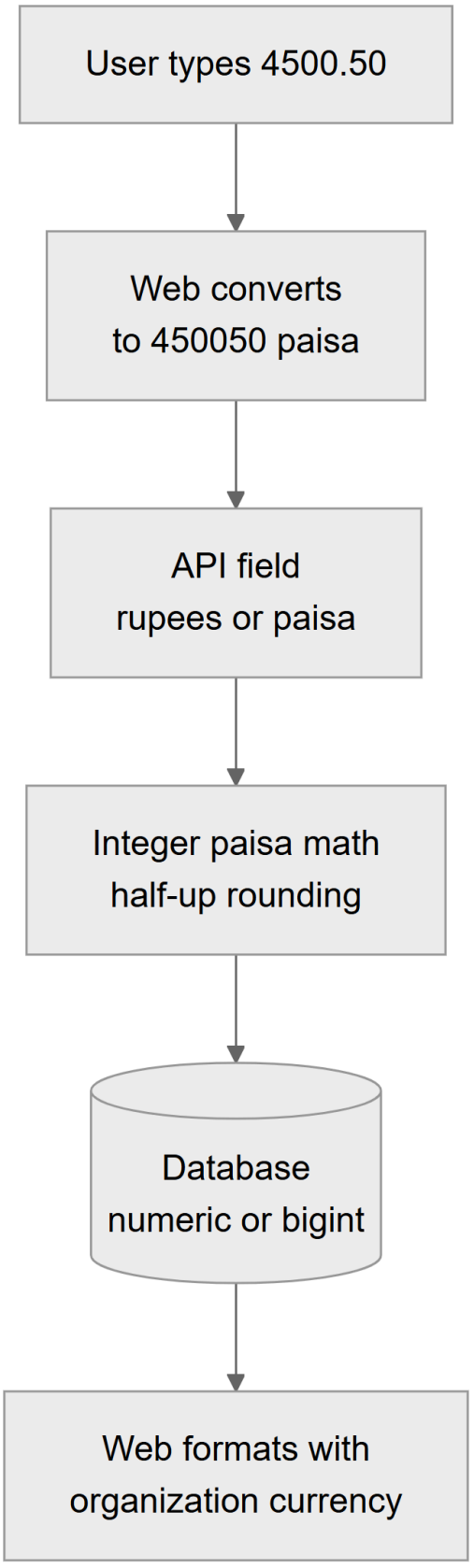


Figure 62.1 — How an amount typed by a person travels to the database and back.

The flowchart reads top to bottom. A user types 4500.50. The web app converts the text to 450050 paisa with integer-only code (no floating point). The API receives the amount as rupees or paisa depending on the field. Server code calculates in integer paisa with half-up rounding. The value is stored in the database as `numeric` or as `bigint` paisa. When the screen reads it back, the web app formats it with the organization's currency.

Tax is exclusive. Prices are before tax. Tax equals the taxable amount (the sum of active charge net amounts after discounts) times the rate, rounded half up once per invoice. The total is subtotal minus discount plus tax. The rate is copied onto the invoice when it is issued from the location's billing settings, and later edits use that copy, not the current setting. Tax is spread over the invoice's charges so that payments can be allocated charge by charge; tax is never revenue and never earns commission. Lab invoices use the same half-up rounding and keep tax in its own paisa column.

Currency. Each organization has a `currency` (default PKR). It is returned with the organization list after sign-in. The web app's `activeCurrency()` and `useCurrency()` read it, and `formatPaisa` writes amounts as "PKR 4,500.50". Number grouping uses a Pakistani locale. The currency is a display label: there is no conversion and no multi-currency ledger.

Technical references: `backend/src/billing/common/money.ts` · `money-column.ts` · `paisa-column.ts` · `backend/src/billing/tax/invoice-tax.ts` · `backend/src/organizations/entities/organization.entity.ts` · `web-app/src/lib/currency.ts` · `web-app/src/features/lab-catalog/lib/money.ts`

62.3 Document numbering

Status: [Partial]. Billing and lab documents are numbered safely; some older numbers are not.

There are four numbering families.

Family	Documents	Format	How it counts
Per location, per type, per year	Patient invoices (`INV`), receipts (`REC`), refunds (`RFD`)	location code, type, year, five digits, for example `MAIN-INV-2026-00001`	Table `document_sequences`
Per organization sequence	Lab order (`LAB`), shipment (`SHP`), lab invoice (`LINV`), lab receipt (`LRC`), payment reversal (`LRV`), credit note (`LCN`), lab refund (`LRF`), vendor bill (`VB`), vendor payment (`VP`)	prefix and at least four digits, for example `LAB-0001`	Table `number_sequences`
Count plus one in application code	Patient number, treatment plan number, patient-page prescription number	`P-00001` , `TP-00001` , `RX-00001`	Number of existing rows plus one
Last number plus one in application code	Purchase order number, prescription number from the prescriptions module	`PO-2026-00001` , `RX-2026-00001`	Reads the latest row and adds one

The table lists the four families with their formats. Invoices, receipts and refunds need the location's **document code** (2 to 6 letters or digits, set in billing settings and locked after the first invoice); without it the Platform refuses to issue the document.

The safe families. The first two families use a database counter row that is locked until the surrounding transaction ends. Concurrent requests wait their turn, no number repeats, and a rollback gives the number back, so numbers have no gaps. The service refuses to run outside a transaction. Invoice sequences restart each year, per location. Lab sequences never restart.

The risky families.

- **Patient, plan and prescription numbers (count plus one).** The next number is the count of rows for the organization plus one. Two registrations at the same moment can compute the same number, and the second save fails on the unique index. Patients are deleted permanently, so after a deletion the count drops and the next number can repeat one that is still in use (the patient count includes soft-deleted rows but not permanently deleted ones). The result in either case is a failed save that the user sees as a generic error, not a wrong document.

- **Purchase order and prescription numbers (last plus one).** The next number reads the most recent row of the organization and adds one, with the year taken from the server clock. Two simultaneous requests can read the same row. In addition, the unique index on the purchase order number and on the prescription number covers all organizations together, so two clinics that both reach the same sequence in the same year would collide.

These are reliability risks, not security ones. A collision stops the save; it does not issue a duplicate.

Technical references: `backend/src/billing/numbering/document-sequence.service.ts` · `DocumentSequenceService` · `backend/src/common/sequences/number-sequence.service.ts` · `NumberSequenceService` · `backend/src/patients/patients.service.ts` · `generatePatientNumber` · `backend/src/inventory/services/purchase-order.service.ts` · `generatePoNumber` · `backend/src/prescriptions/services/prescription.service.ts` · `generatePrescriptionNumber`

62.4 Limitations and known gaps

- Several jobs and statistics use the server timezone, not the clinic's.
- Two storage styles for money (decimal rupees and integer paisa) exist side by side; conversion happens at module boundaries.
- Count-based and last-number-based document numbers can collide under concurrency, and purchase order and prescription numbers are unique across all clinics together.
- No multi-currency support or conversion.
- Tax is exclusive only; there is no tax-inclusive pricing.

62.5 Owner's FAQ

What time does "today" mean for my branch? The branch's own timezone, set on the location.

Why can a daily job run at an odd hour for my clinic? Jobs follow the server's timezone. A job set for 01:00 runs at 01:00 server time.

Can an invoice number repeat? No. Invoice, receipt and refund numbers come from a locked counter per branch and year.

Can two patients get the same patient number? The database refuses a duplicate, so the second save fails and must be tried again.

Are my prices with or without tax? Without. Tax is added on top at the rate set for the branch when the invoice is issued.

Can I invoice in another currency? You can set the organization's currency label, but there is no conversion.

Chapter 63 — Realtime updates and background jobs

This chapter explains the two ways the Platform acts without a user clicking: it pushes live updates to open screens over a socket connection, and it runs timed jobs and background loops in a worker process. It is written for the product owner who must understand which processes to run and what happens when Redis or the worker is down. Appendix E holds the full job table; this chapter restates the essentials and explains how the pieces fit together. The queue module (Chapter 30) and notifications (Chapters 44 and 59) are the main users of these mechanisms.

63.1 Process roles

Status: [Implemented]

The backend is one program that starts in one of three roles, chosen by the start command.

Role	Start file	Serves web requests and sockets	Runs jobs and relays	Needs Redis
`api`	`main.api.ts`	Yes	No	Yes, to receive events
`worker`	`main.worker.ts`	No	Yes	Yes, to send events
`all`	`main.ts`	Yes	Yes	No

The table shows the three roles. In production, run two or more `api` processes behind a load balancer and **exactly one** `worker`. In development, the single `all` process does everything and needs no Redis, because the relay hands events directly to the local socket gateway. The job scheduler is enabled only in processes whose role is not `api`, so adding API processes never duplicates a job.

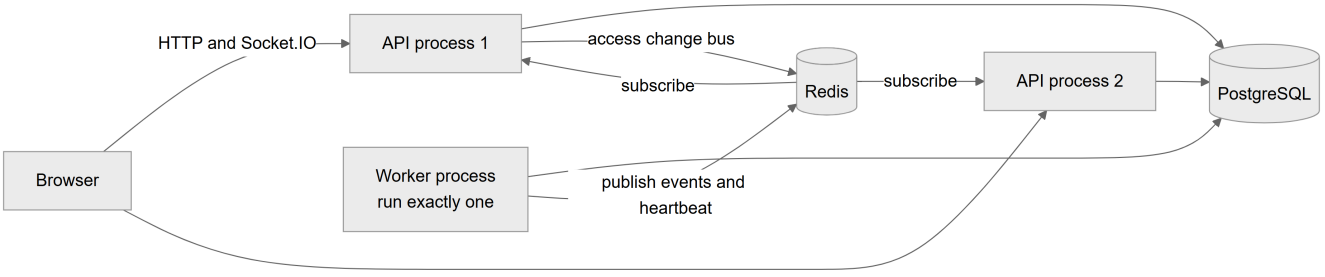


Figure 63.1 — The browser, two API processes, one worker, PostgreSQL and Redis.

The flowchart reads left to right. The Browser talks to API process 1 and API process 2 over HTTP and Socket.IO. Both API processes read and write PostgreSQL. The Worker process, marked "run exactly one", also uses PostgreSQL. The worker publishes events and a heartbeat to Redis. Redis delivers them to both API processes through a subscription. A further arrow shows an API process using Redis for the access change bus, which is how one API process tells the others that access changed (Chapter 57).

Technical references: `backend/src/common/runtime/process-role.ts` · `backend/src/main.ts` · `main.api.ts` · `main.worker.ts` · `backend/src/app.module.ts` · `AppModule.forRole`

63.2 The realtime gateway

Status: [Implemented]

What it does. Pushes small "something changed" messages to open browser screens so that the reception board, doctor screen, waiting-room display and permission-aware screens stay live without constant refreshing.

How it connects. A Socket.IO gateway listens at the path `/realtime` of the API. The web app connects with websocket transport and passes its access token in the handshake. The token is verified once, when the socket connects. A browser with no token may connect anonymously, but may join only the public display room. After connecting, the client sends a `subscribe` message naming a room; the server answers with success or a reason.

Rooms.

Room	Who may join	What it carries
`session:<id>`	Users with access to the queue session's location	Events for one doctor's queue (doctor screen, reception)
`location:<id>:board`	Users with access to the location	Events of every queue at the location (reception board)
`location:<id>:display`	Anyone, no sign-in	Waiting-room TV events; payloads carry token numbers and doctor names, never patient names
`org:<id>`	Users with access to the organization (super admin, owner, or an active staff assignment)	The `access.changed` event with the organization id and the new version

The table lists the four room types. Anything else gets "Unknown room"; a room the user may not enter gets "No access to this room".

Events are nudges. A message carries a sequence number (`seq`). The browser keeps the sequence of the snapshot it loaded, ignores older or duplicate messages and fetches a fresh snapshot when it sees a newer one. This makes delivery at least once safe, because a duplicate does nothing.

Technical references: `backend/src/queue/realtime/queue.gateway.ts` · `QueueGateway` · `web-app/src/features/access/hooks/useAccessSocket.ts`

63.3 From a commit to a screen: outbox, relay, Redis

Status: [Implemented]

When a queue action changes state, the same database transaction also writes an event row in the `outbox` table and sends a database NOTIFY, so an event exists only if the change was committed. The **outbox relay** in the worker takes unpublished rows (up to 100 at a time, locking them so a second relay would skip them), publishes each to one Redis channel, waits until Redis accepted the whole batch and only then marks the rows as published. If Redis is down, the batch is rolled back and tried again every second. It is woken by the NOTIFY and also polls every second as a fallback. Every API process subscribes to the channel and emits each message to its own local sockets, so a browser connected to any API process sees every event. Chapter 5, section 5.8, shows this as a sequence diagram.

Heartbeat and watchdog. The worker publishes a heartbeat to the same channel every 5 seconds. Each API process runs a watchdog: it considers the bus healthy only while it is subscribed and has seen a heartbeat within 15 seconds. When the bus is unhealthy (Redis or worker down), the API process **refuses new socket connections and drops existing ones**, using a transport close so that browsers keep trying to reconnect. Screens then fall back to polling about every 10 seconds and reload their data once the socket returns. At start-up the gateway stays closed until the first heartbeat arrives.

Access changes. The access change bus is a second, separate path on its own Redis channel. When an organization's access version is bumped, the bus notifies the local caches and the local gateway, and publishes to Redis so the other processes do the same. Each process then emits `access.changed` to its sockets in the `org:<id>` room. This path does not use the outbox or the worker (Chapter 57).

Single-process mode. In the `all` role, the relay publishes straight to the local gateway.

Technical references: `backend/src/queue/realtime/outbox-relay.service.ts` · `OutboxRelayService` · `redis-realtime.publisher.ts` · `redis-realtime.subscriber.ts` · `realtime-bus.ts` · `in-process-realtime.publisher.ts` · `backend/src/access/version/access-change-bus.ts`

63.4 Scheduled jobs

Status: [Implemented] (the inventory alert job is [Partial])

All jobs run in the worker (or the `all` process). The schedules are in **server time**; none sets a timezone.

Job	Schedule	What it does
Queue no-show marker	every 60 seconds	Marks a booked patient as no-show once they have not checked in by the start time plus the location's maximum lateness
Queue call timeout	every 15 seconds	Skips a called patient who never came, when the queue policy sets a timeout
Queue session day-end	every 5 minutes	Closes empty queue sessions of a past clinic day; leaves non-empty ones open and logs an alert
Queue cleanup	daily at 03:15	Deletes queue idempotency keys older than 24 hours and published outbox rows older than 7 days
Patient credit expiry	daily at 01:00	Marks prepaid credits past their expiry date as expired
Billing invariant check	daily at 02:30	Recomputes billing totals, checks money rules including the commission ledger and logs any break
Month-end payroll	05:00 on the 1st of each month	Generates payroll records for the month just ended for every active location; locations already generated are skipped
Inventory alerts	daily at 08:00	Counts low-stock and soon-expiring items and writes a warning to the server log only

The table lists the eight timed jobs. The first four keep the live queue consistent; the others handle daily and monthly housekeeping. Appendix E gives the same table with statuses and references. The queue day-end job is the only one that respects each location's clinic day, taken from its queue policy.

Background loops. Three loops run continuously in the worker. The **queue outbox relay** (section 63.3). The **notification relay**, which is woken by a database NOTIFY and polls every two seconds, sends notification rows to the inbox (Chapter 59). The **heartbeat** is published every 5 seconds. Both relays follow the outbox pattern: a change and its message commit together, and a separate loop delivers the message later. If the database NOTIFY connection drops, each relay logs a warning and keeps working by polling.

Other timers. The in-memory rate limit tables in the authentication guards and the magic-link throttle are swept every minute or when they grow large; they do not need the worker.

Technical references: `backend/src/queue/jobs/queue-jobs.service.ts` · `backend/src/billing/patient-credits.service.ts` · `backend/src/billing/invariants/billing-invariant.job.ts` · `backend/src/payroll/payroll.service.ts` · `backend/src/inventory/services/inventory-alerts.service.ts` · `backend/src/notifications/delivery/notification-relay.service.ts` · `docs/handbook/src/appendix/appE-scheduled-jobs.md`

63.5 Limitations and known gaps

- Exactly one worker must run; nothing in the code stops a second worker, and a second worker would run each timed job a second time (the relays are safe with several workers, the timed jobs are not guaranteed to be).
- Job times follow the server timezone, not the clinics'.
- Inventory alerts reach only the server log, not any user.
- Realtime depends on Redis in production; without it screens poll about every 10 seconds.
- The access token is checked once when a socket connects; the connection continues after the token expires.
- There is no clean-up job for session rows, security logs or the notification outbox.

63.6 Owner's FAQ

What happens if Redis is down? Realtime stops: the API refuses sockets and screens poll every ten seconds, so they update a little later. Access changes still reach the process that made them at once and the others within 15 seconds.

What happens if the worker is down? No timed jobs run, no notifications are sent and no queue events are published. Rows wait in the database and are processed when the worker returns. Screens fall back to polling.

Can I run two workers for safety? No. Run exactly one. A second worker would repeat the scheduled jobs.

Do I need Redis on my laptop? No. The single `all` process does not need it.

Can the waiting-room TV show patient names? No. The public display room carries token numbers and doctor names only.